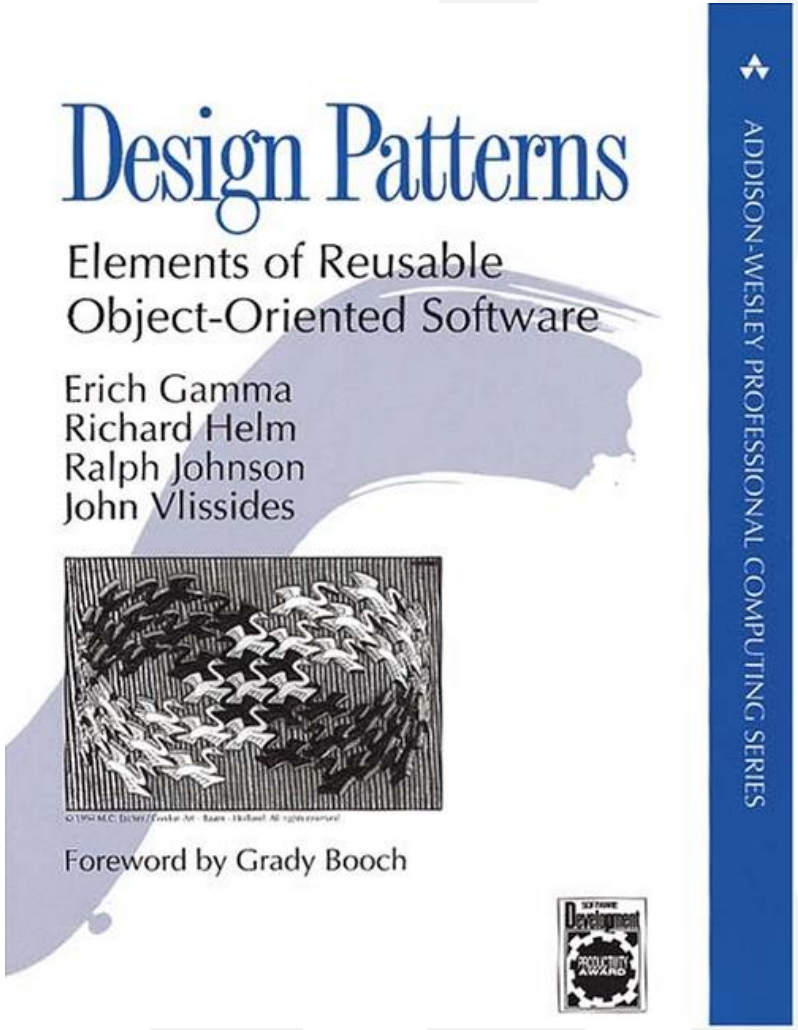
Design Patterns



An architecture helps with

System understanding: interactions between modules

Reuse: high-level view shows opportunity for reuse

Construction: breaks development down into work items; provides a path from requirements to code

Evolution: high-level view shows evolution path

Management: helps understand work items and track progress

Communication: provides vocabulary; a picture says 1000 words

Properties of architecture

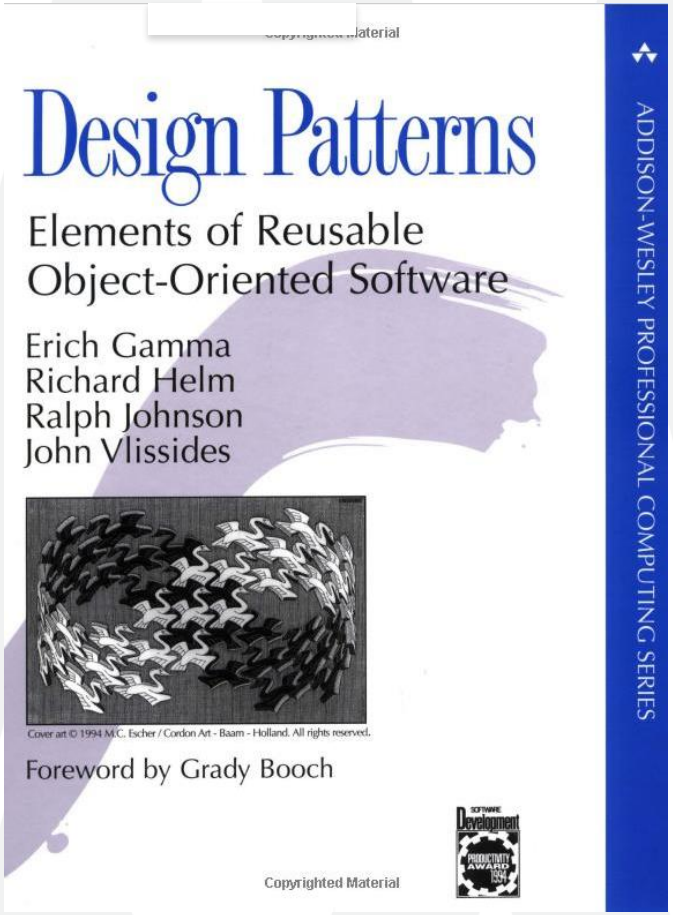
- Coupling
- Cohesion
- Style conformity
- Matching
- Errosion

Architectural Styles

- Software can display many architectural styles.
  - **A set of components** (e.g.: a database, computational modules) that will perform a function required by the system.
  - **The set of connectors** will help in coordination, communication, and cooperation between the components.
  - **Conditions** that how components can be integrated to form the system.
  - **Semantic models** that help the designer to understand the overall properties of the system.

The Gang of Four (GoF)

- 23 design patterns
- Description of communicating objects & classes
  - Captures common (and successful) solution to a *category* of related problem instances
  - Can be customized to solve a specific (new) problem in that category



The GoF Pattern Zoo

Creational

1. Factory
2. ***Abstract Factory***
3. Builder
4. Prototype
5. ***Singleton/Null Obj***

Structural

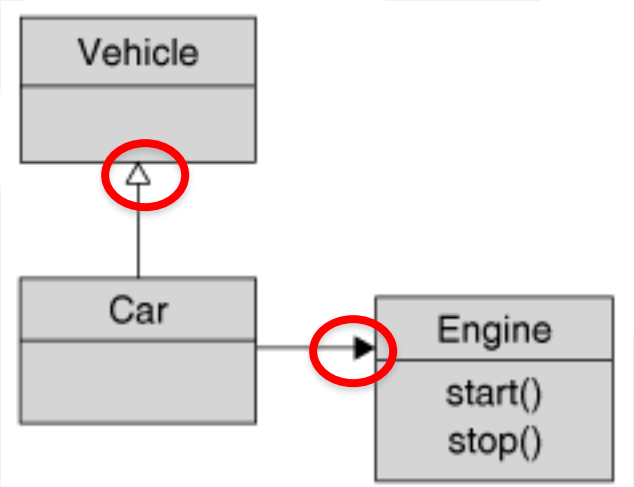
6. ***Adapter***
7. ***Composite***
8. ***Proxy***
9. Bridge
10. Flyweight
11. ***Façade***
12. ***Decorator***

Behavioral

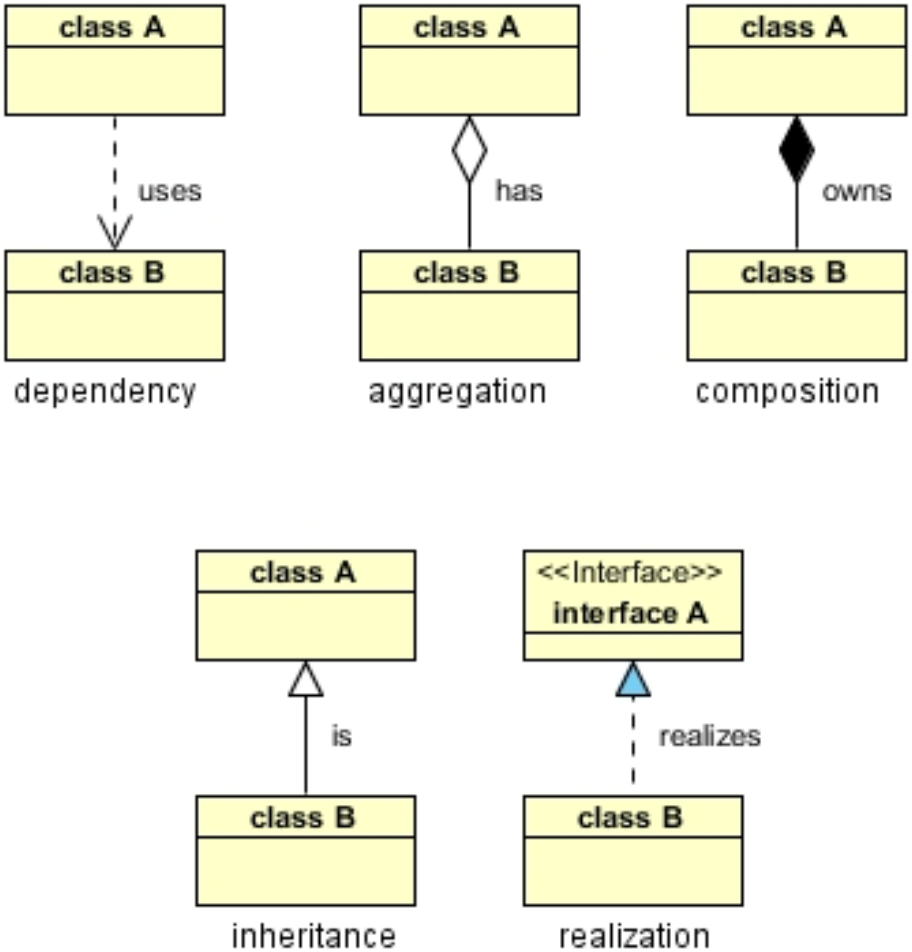
13. ***Observer***
14. Mediator
15. Chain of responsibility
16. Command
17. Interpreter
18. ***Iterator***
19. Memento (memoization)
20. State
21. ***Strategy***
22. ***Template***
23. Visitor

Just Enough UML

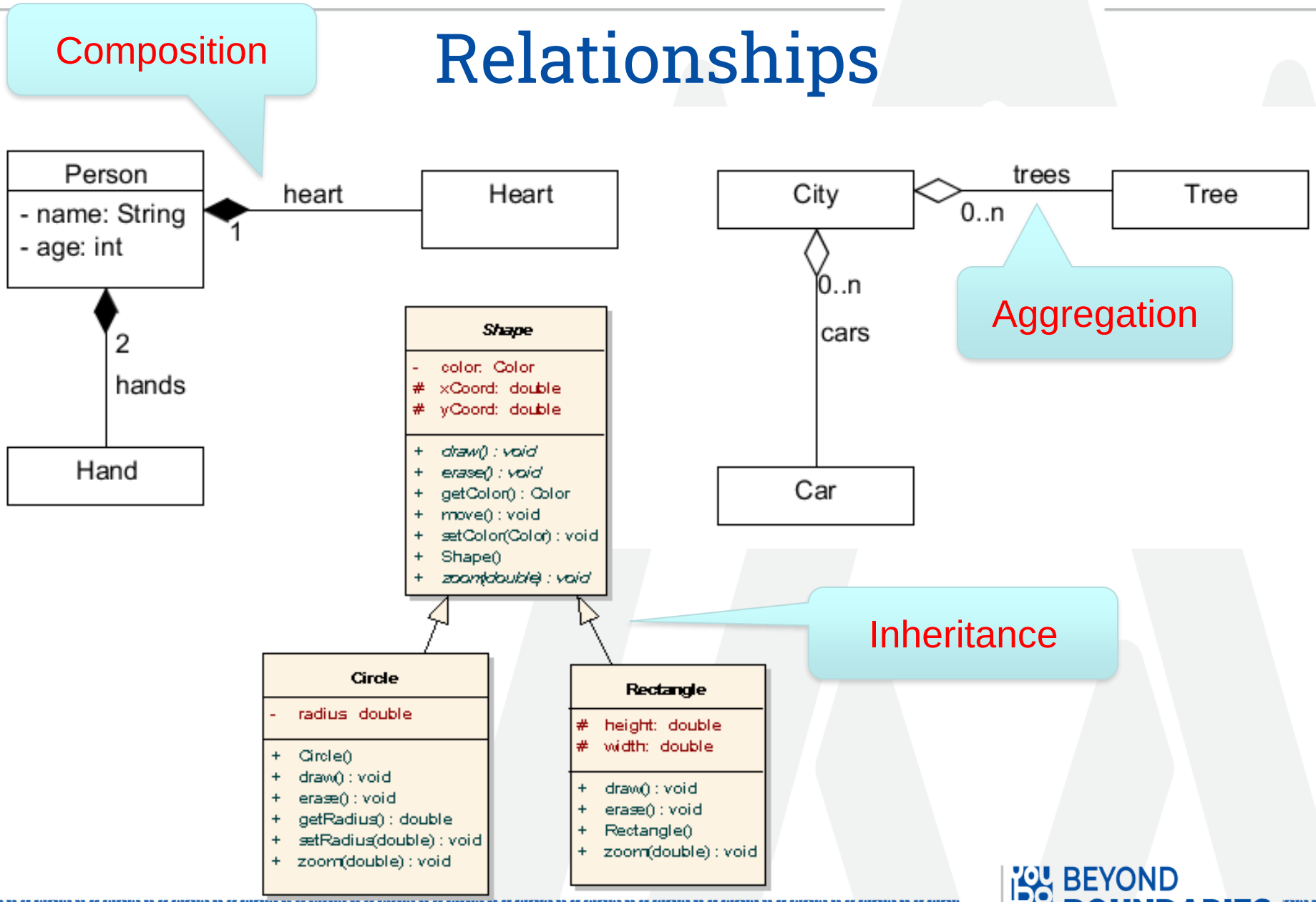
- **Unified Modeling Language (UML)**: notation for describing various artifacts in OOP systems
- One type of UML diagram is a *class diagram*, showing class relationships and principal methods:
  - `Car` is a subclass of `Vehicle`
  - `Engine` is a *component* of `Car`
  - `Engine` includes `start()`, `stop()` methods



Relationships



Relationships



Design Patterns Classification

- **Creational patterns:**
  - Deal with initializing and configuring objects
- **Structural patterns**:
  - Composition of classes or objects
  - Decouple interface and implementation of classes
- **Behavioral patterns**:
  - Deal with dynamic interactions among societies of objects
  - How they distribute responsibility

elements

- Design patterns have 4 essential elements:
  - **Pattern** name: increases vocabulary of designers
  - **Problem**: intent, context, when to apply
  - **Solution**: UML-like structure, abstract code
  - **Consequences**: results and tradeoffs

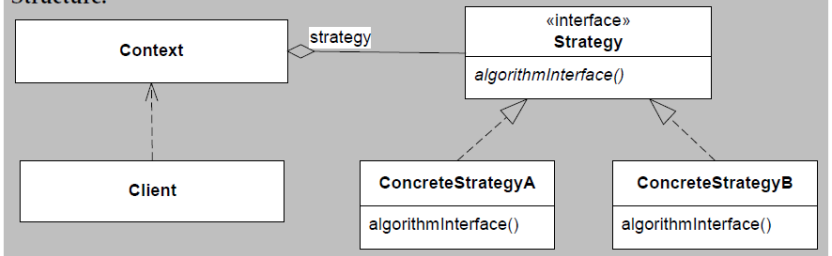
[7.1] Design Pattern: Strategy

Intent Define a family of business rules or algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithms vary independently from clients that use it.

Problem Your product must support variable algorithms or business rules and you want a flexible and reliable way of controling the variability.

Solution Separate selection of algorithm from its implementation by expressing the algorithms responsibilities in an interface and let each implementation of the algorithms realize this interface.

Structure:



Roles **Strategy** specifies the responsibility and interface of the algorithm. **ConcreteStrategies** defines concrete behavior fulfilling the responsibility. **Context** performs its work for **Client** by delegating to an instance of type **Strategy**.

Cost - Benefit The benefits are: *Strategies eliminate conditional statements.* It is an *alternative to subclassing*. It facilitates *separate testing* of **Context** and **ConcreteStrategy**.
The liabilities are: *Increased number of objects. Clients must be aware of strategies.*

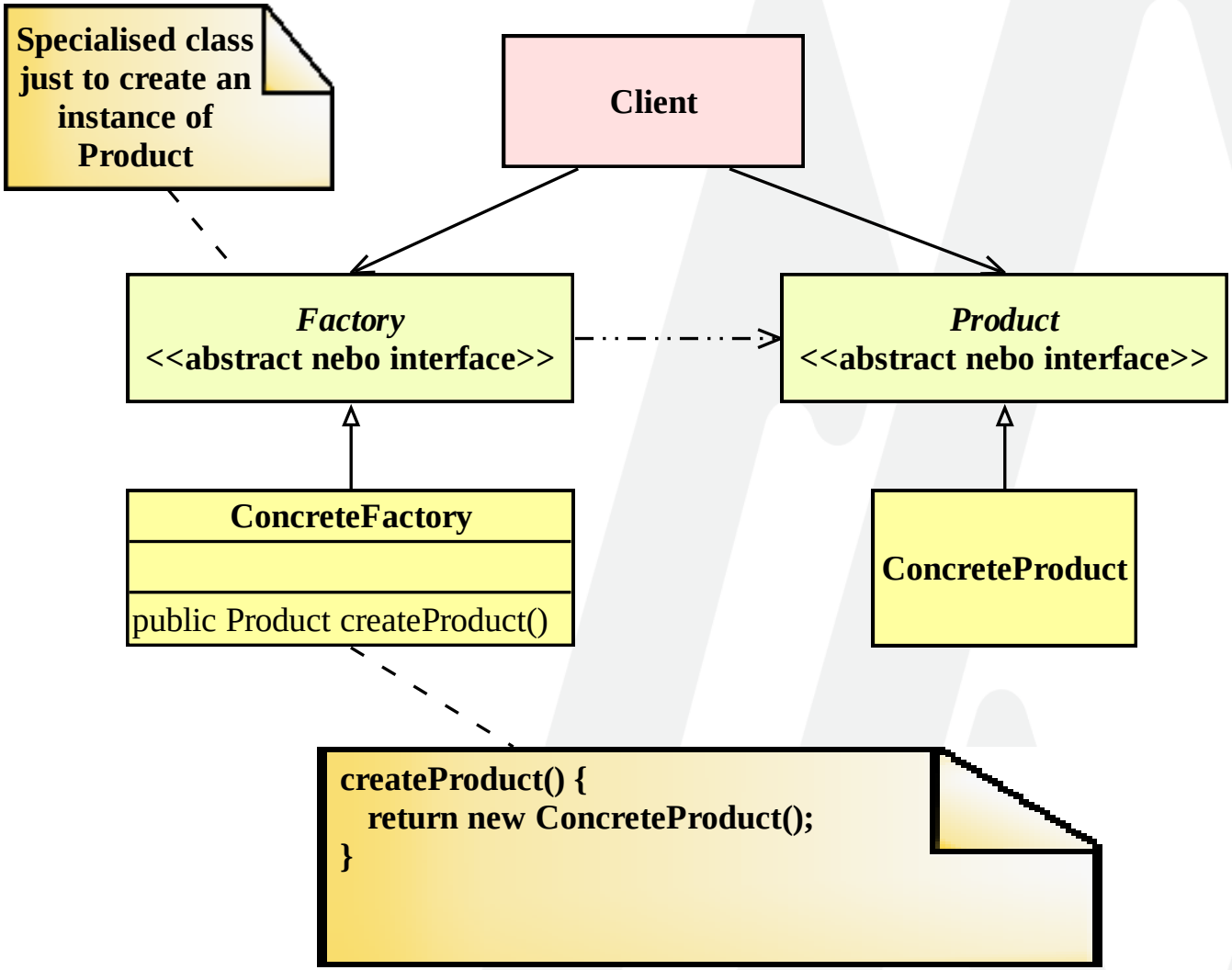
Creational Design Patterns

- Singleton
- Factory Method
- Factory Pattern
- Abstract Factory
- Builder
- Reusable Pool
- Prototype

Singleton

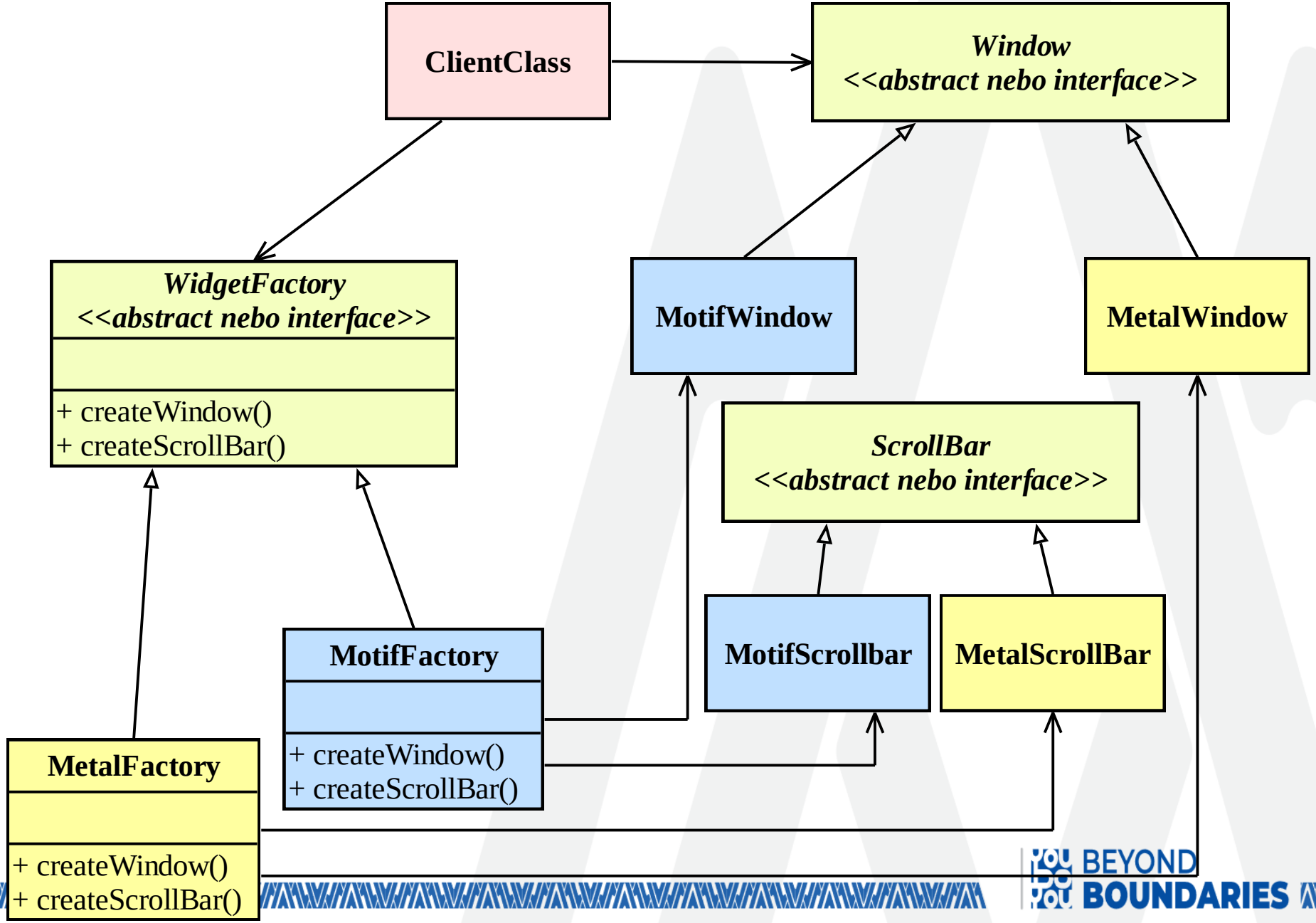
```
Class Singleton {
    private static Singleton instance = null;

    private Singleton() {}; //private constructor

    public static
    synchronized
    Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
```

Abstract Factory Pattern

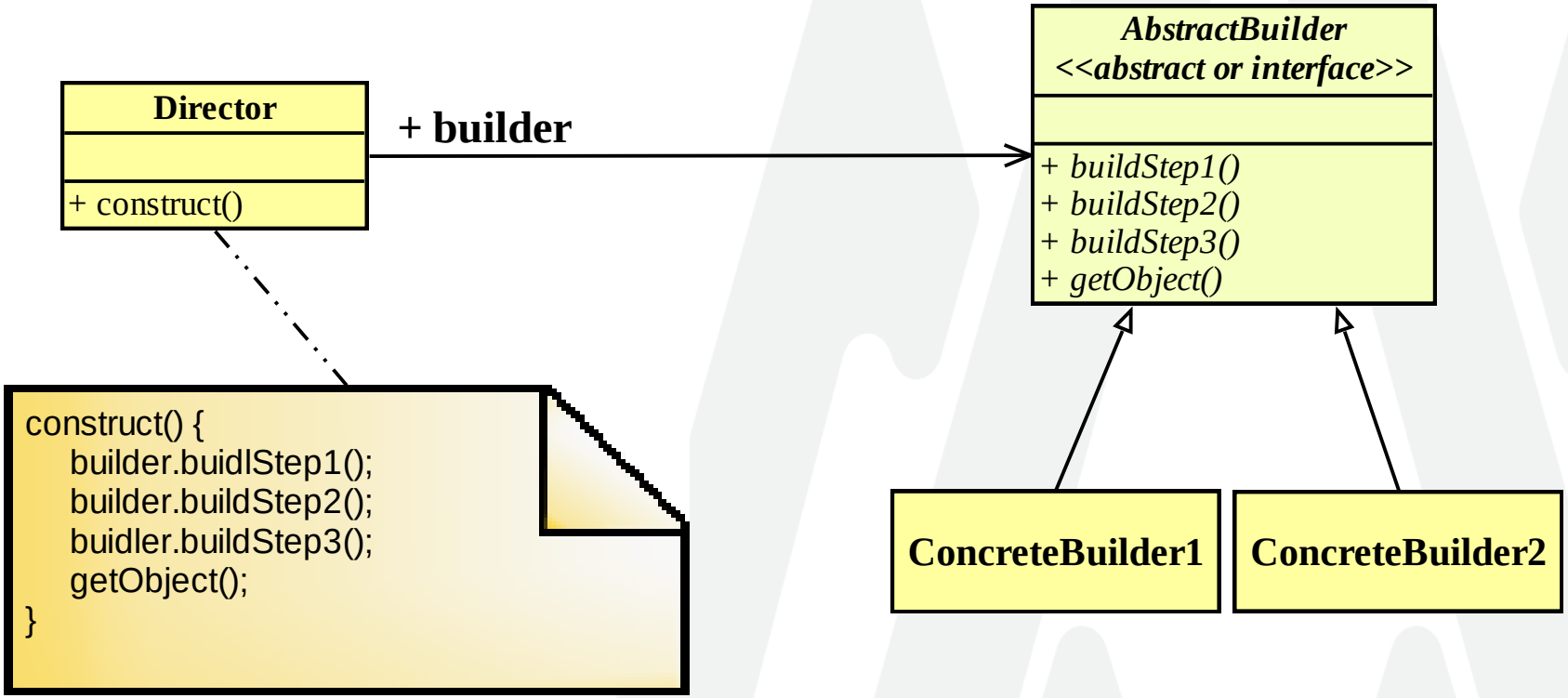


BEYOND BOUNDARIES

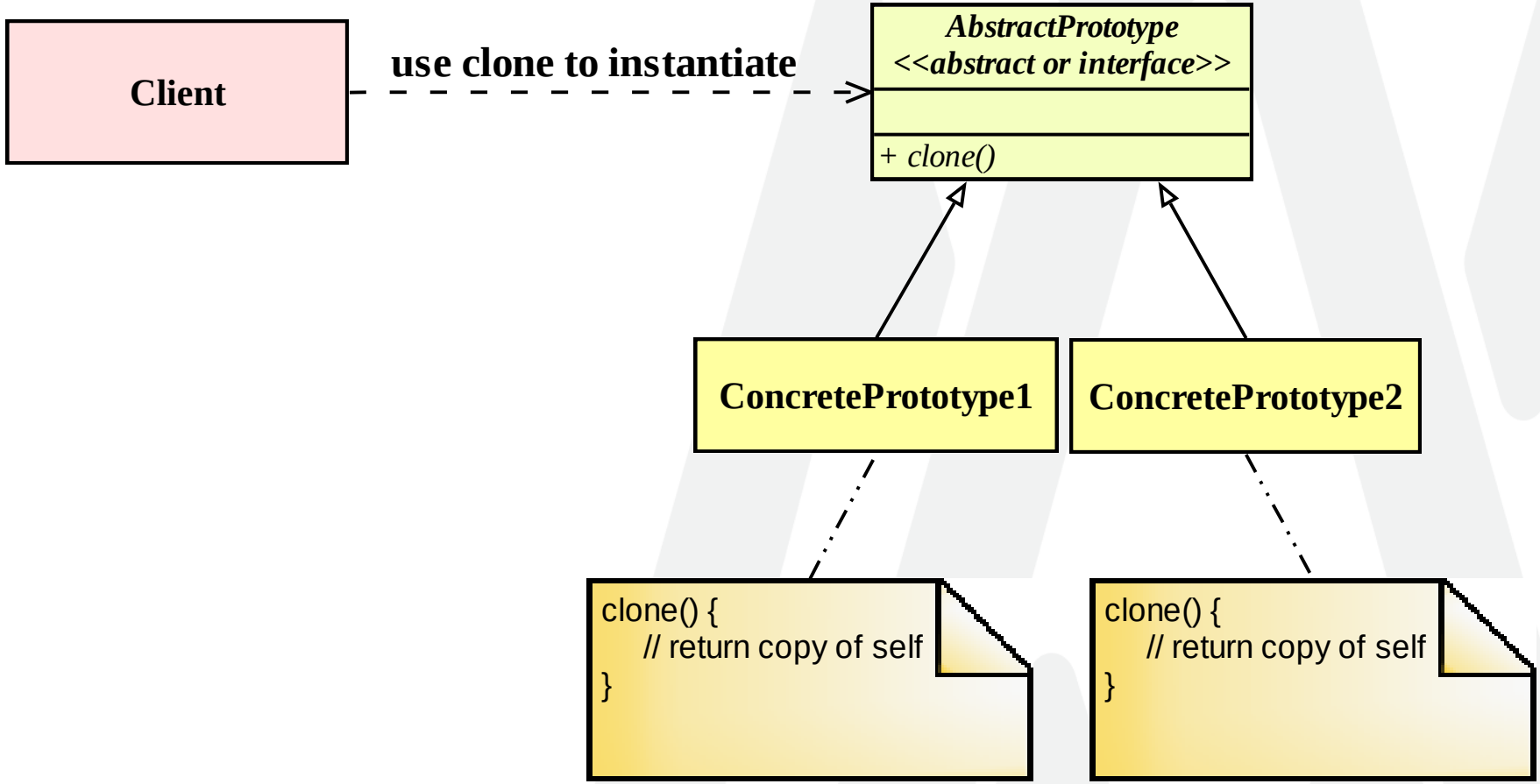
Abstract Factory Pattern



Builder

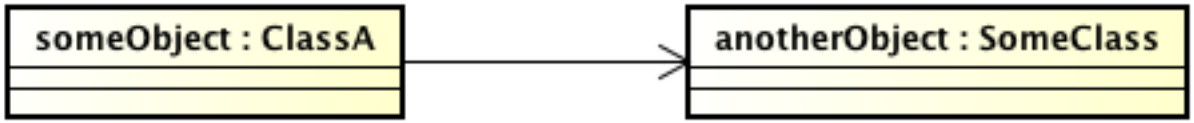

Director
+ construct()
+ builder
AbstractBuilder
<<abstract or interface>>
+ buildStep1()
+ buildStep2()
+ buildStep3()
+ getObject()
construct() {
builder.buidlStep1();
builder.buildStep2();
buidler.buildStep3();
getObject();
}
ConcreteBuilder1
ConcreteBuilder2

Prototype


Client
use clone to instantiate
AbstractPrototype
<<abstract or interface>>
+ clone()
ConcretePrototype1
ConcretePrototype2
clone() {
// return copy of self
}
clone() {
// return copy of self
}

Prototype

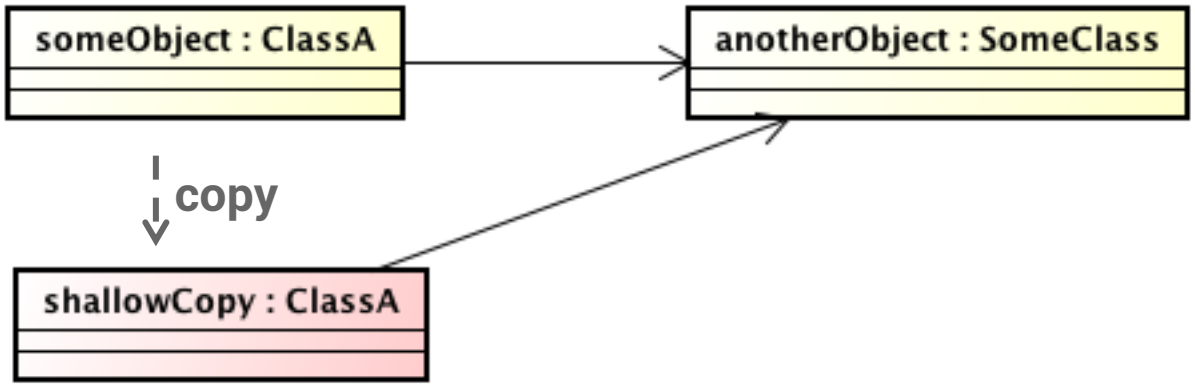
Shallow copy



BEYOND BOUNDARIES

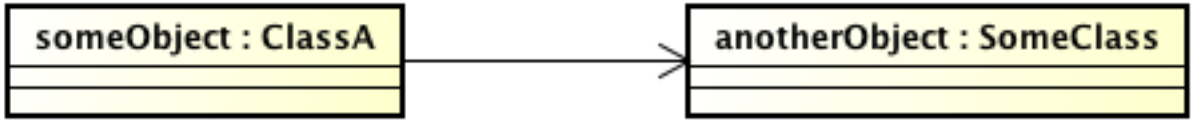
Prototype

Shallow copy



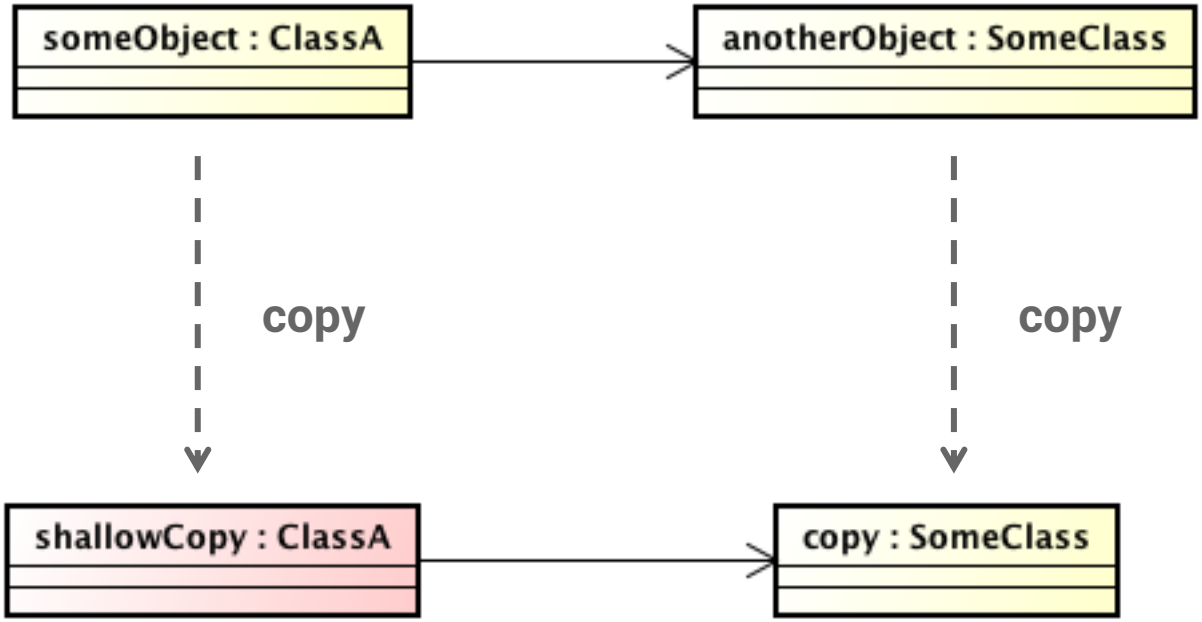
Prototype

Deep copy



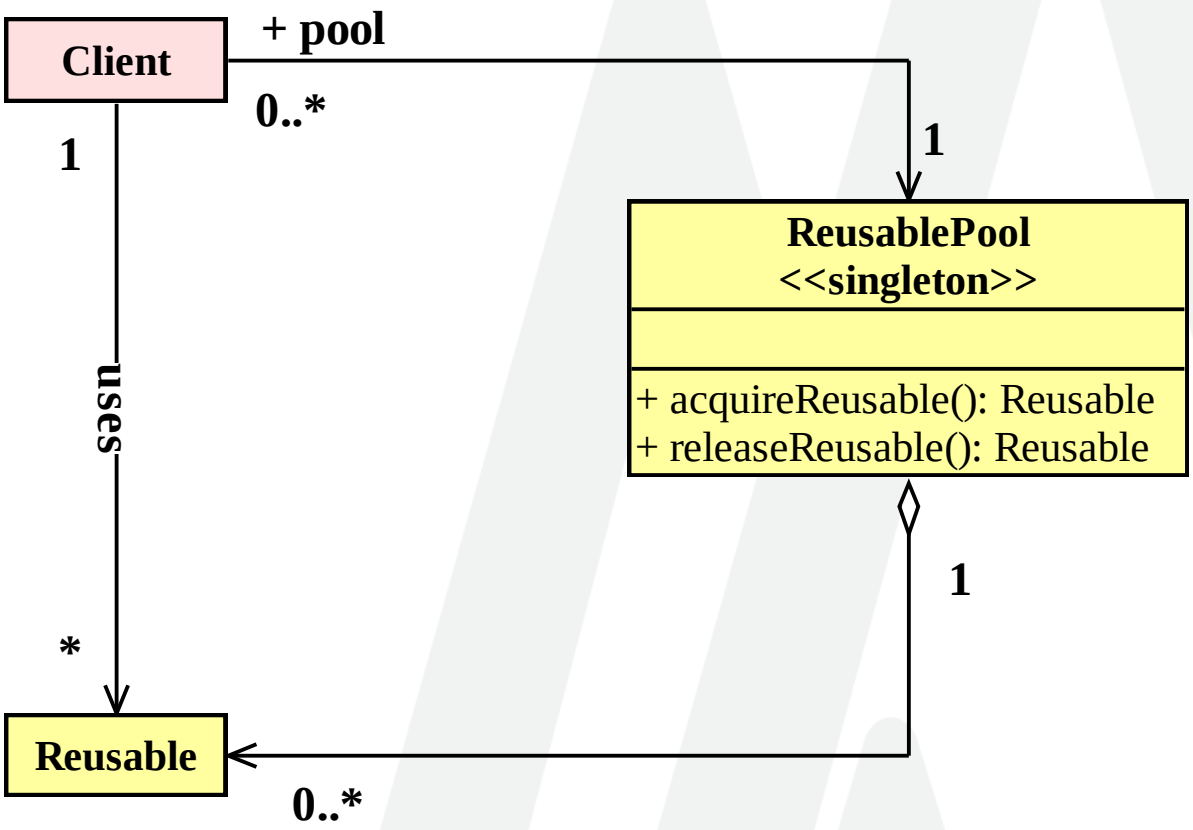
Prototype

Deep copy



BEYOND BOUNDARIES

Reusable Pool

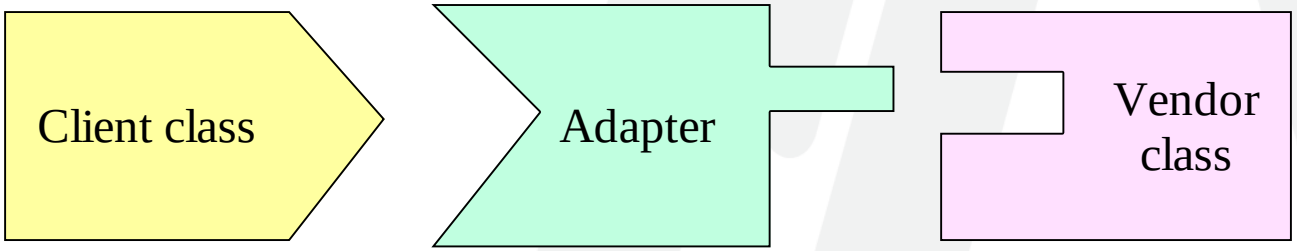


Structural Design Patterns

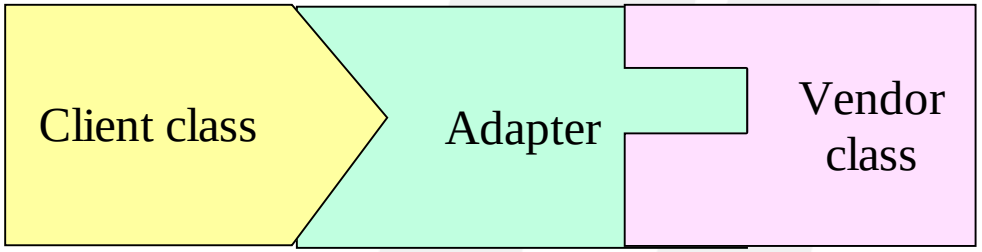
- Adapter
- Bridge (sometimes considered to be a behavioral DP)
- Decorator
- Composite
- Facade
- Proxy
- and more

Adapter

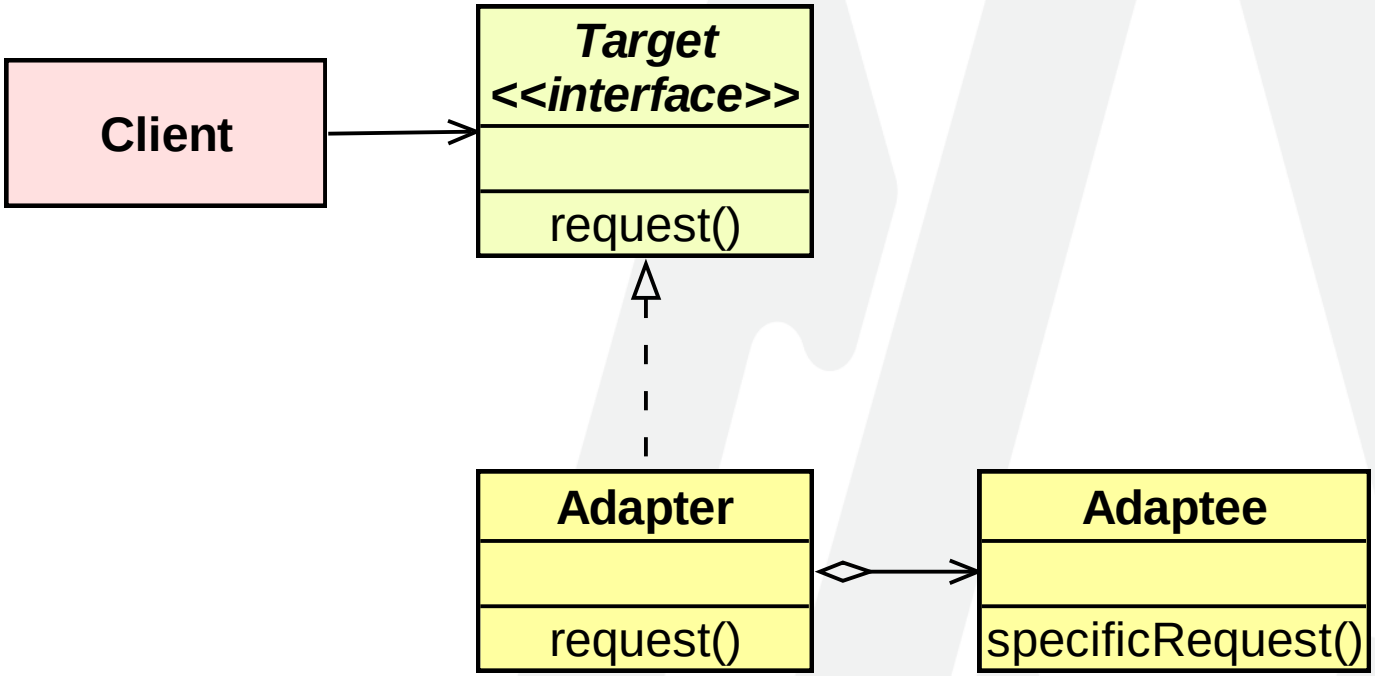
Adapter



Adapter

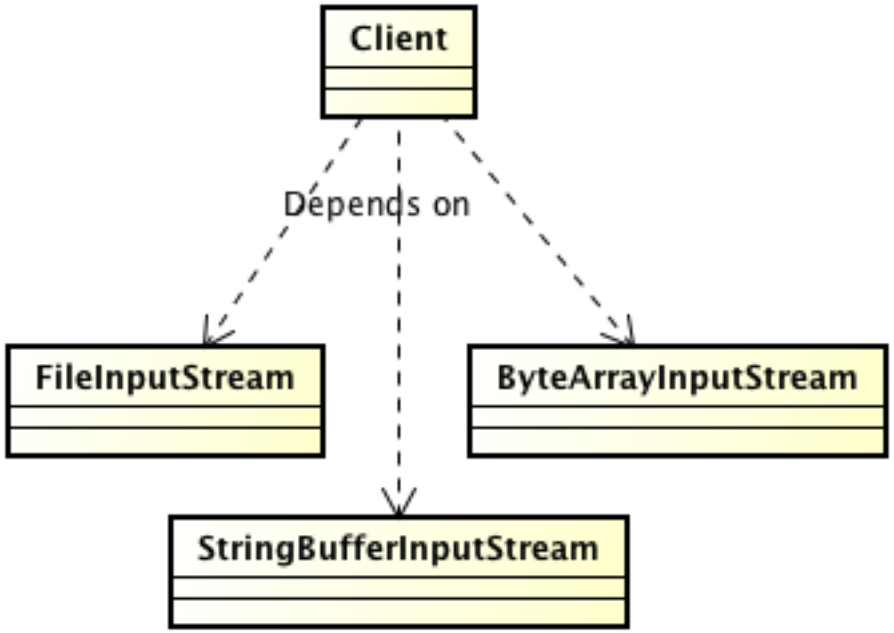


Adapter


Client
Target
<<interface>>
request()
Adapter
request()
Adaptee
specificRequest()

The Dependency Inversion Principle

- High level modules should not depend upon low level modules; both should depend on abstractions.
- Abstractions should not depend upon details; details should depend upon abstractions.



Dependency Inversion Principle

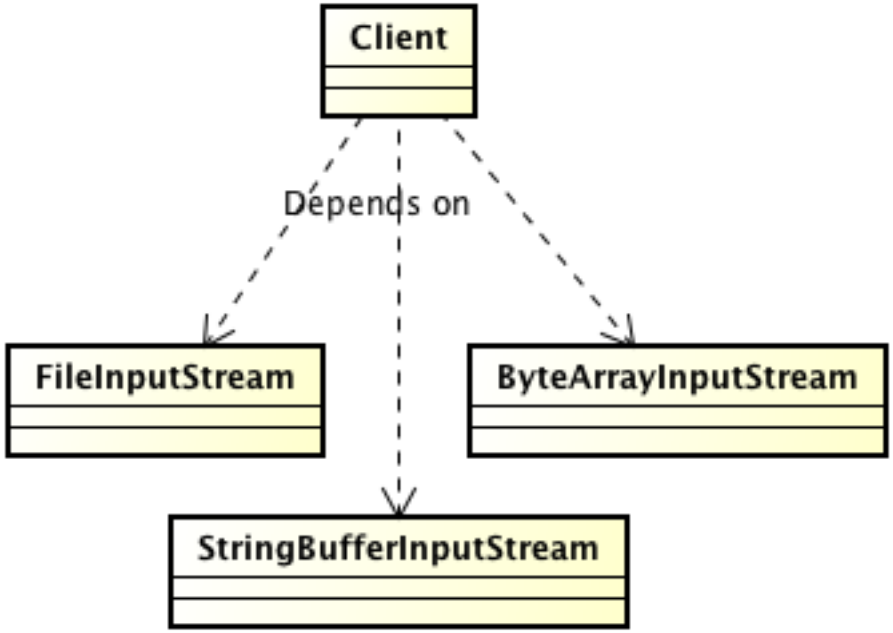
Dependency inversion principle

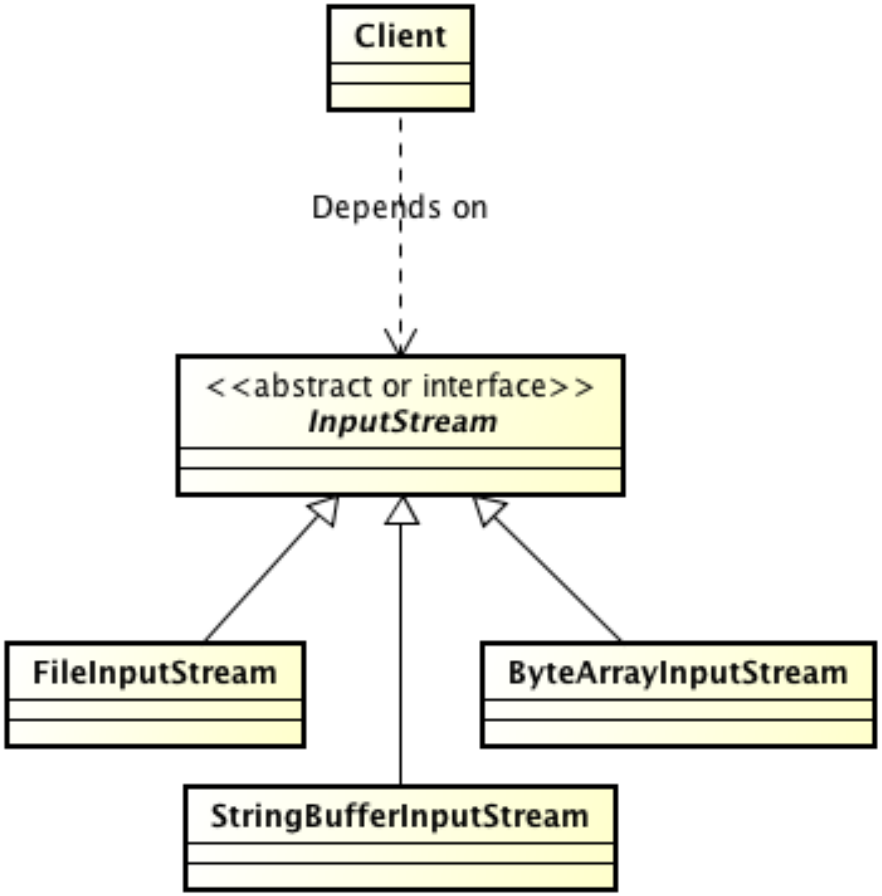
specific form of decoupling where conventional dependency relationships established from high-level are inverted (e.g. reversed) for the purpose of rendering high-level modules independent of the low-level module implementation details. The principle states:

- *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
- *Abstractions should not depend upon details. Details should depend upon abstractions.*

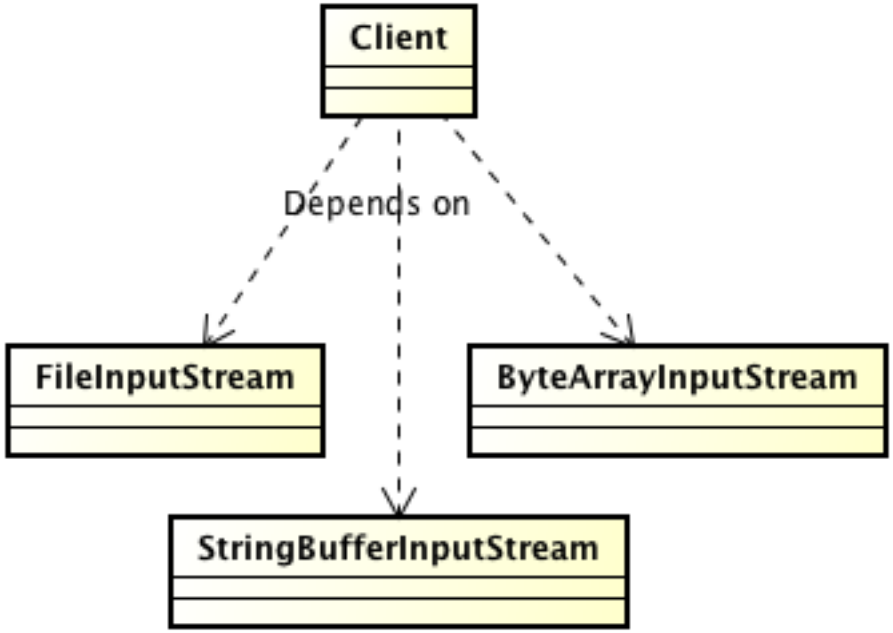
(From Wikipedia, the free encyclopedia)

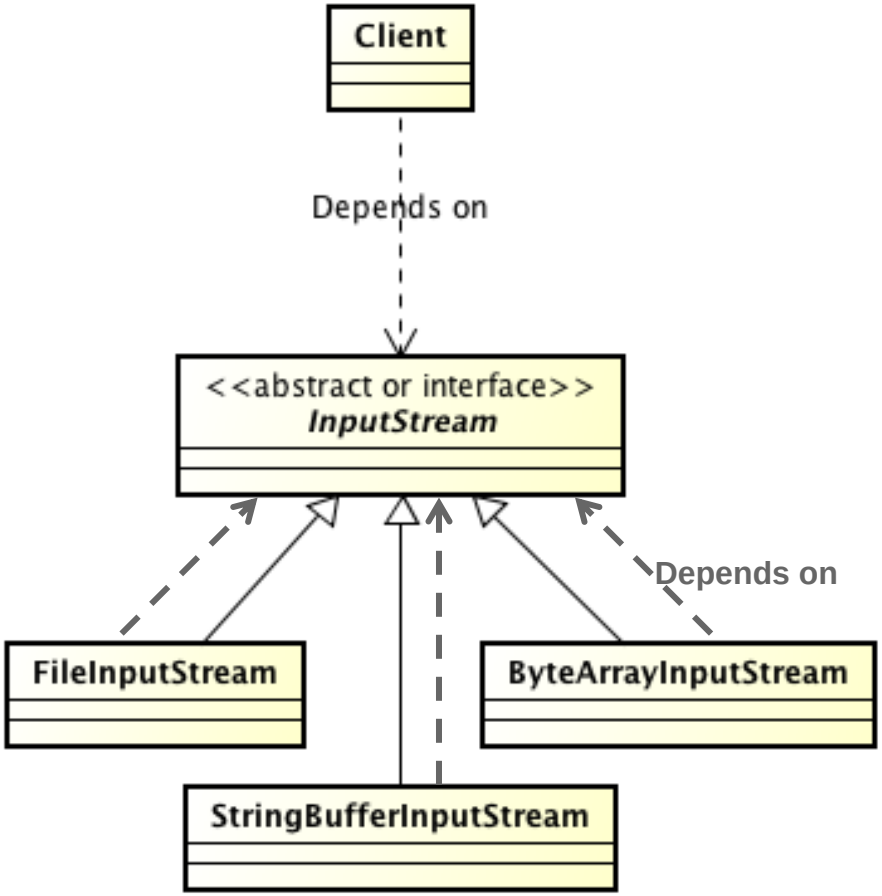
The Dependency Inversion Principle



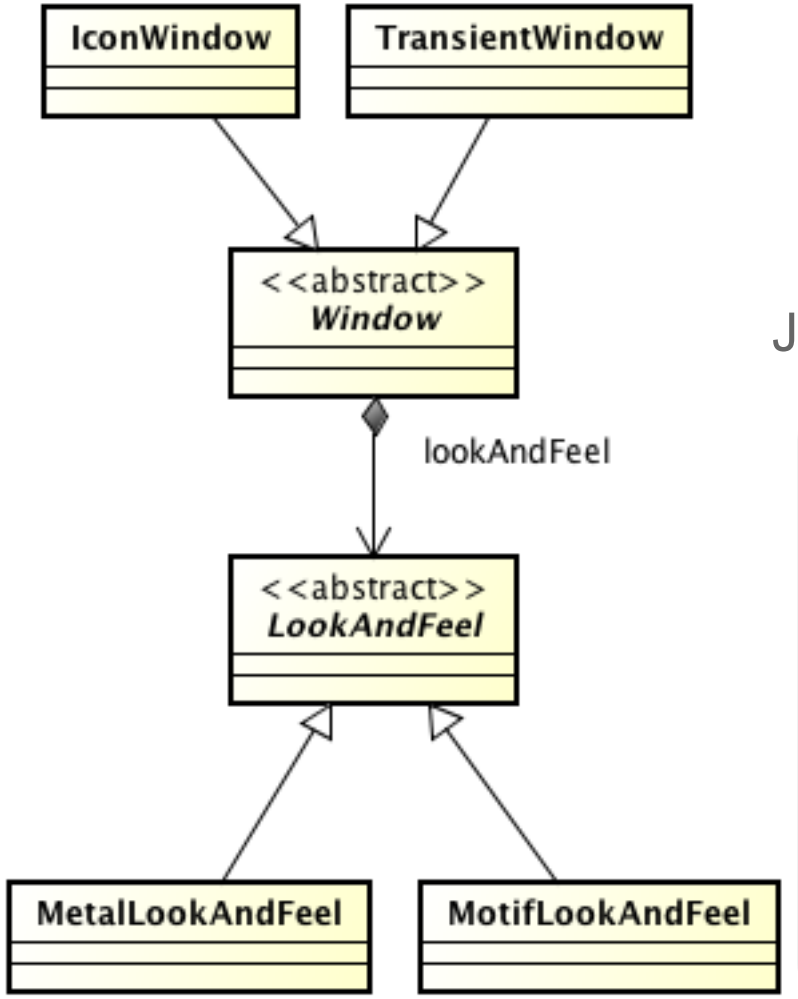


The Dependency Inversion Principle





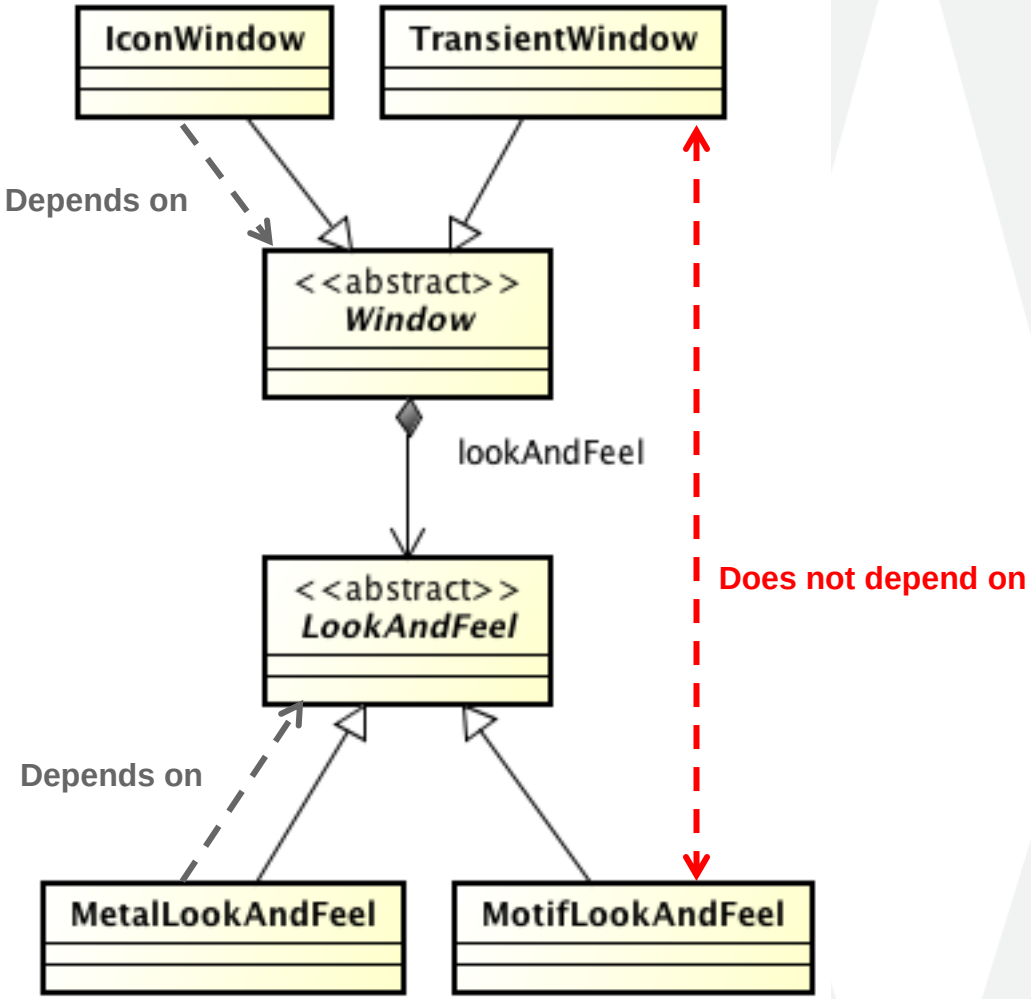
Bridge



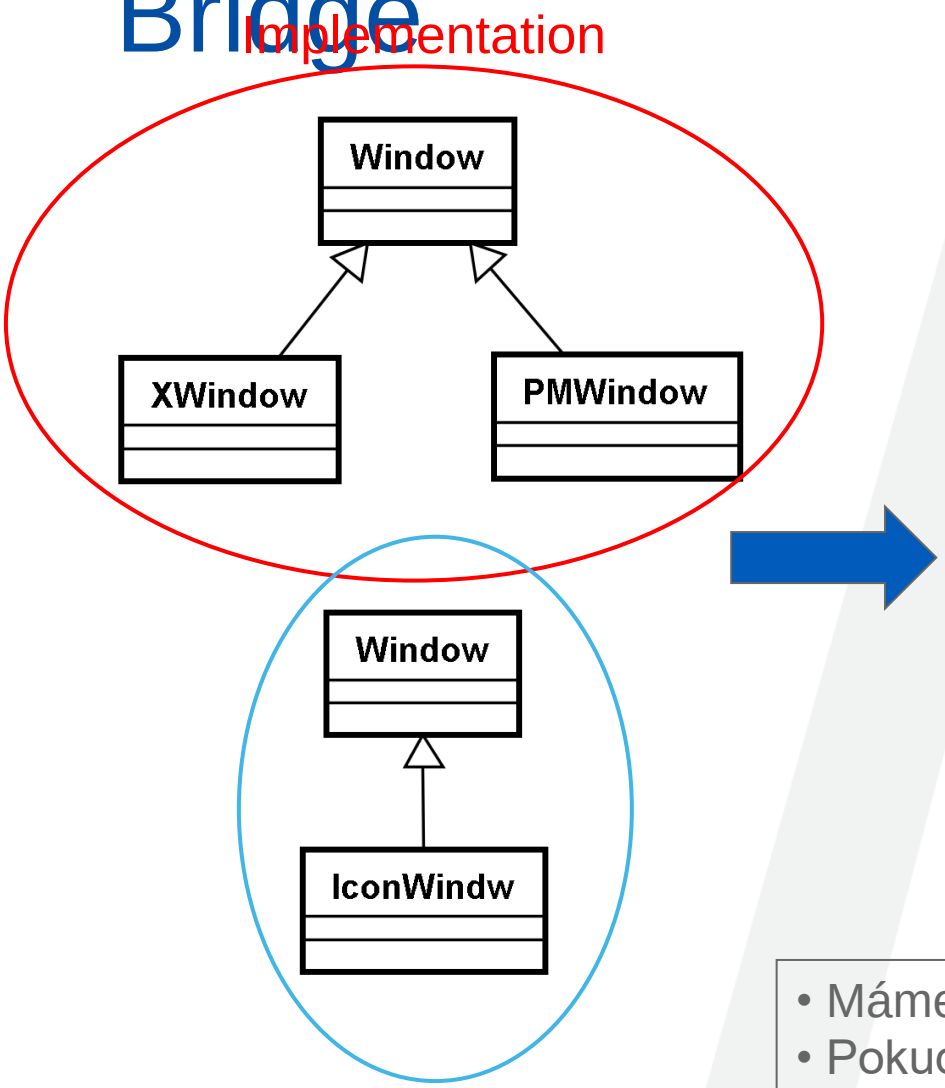
Jednoduché rozšíření předešlého diagramu

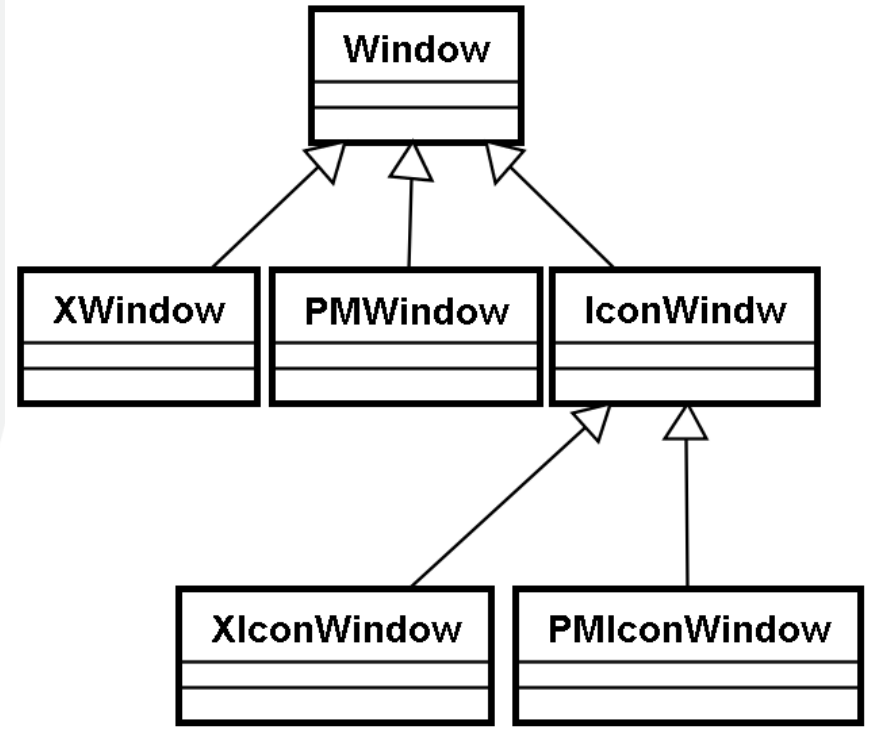
- design pattern Bridge

Bridge


IconWindow
TransientWindow
Depends on
<<abstract>>
Window
lookAndFeel
<<abstract>>
LookAndFeel
Does not depend on
Depends on
MetalLookAndFeel
MotifLookAndFeel

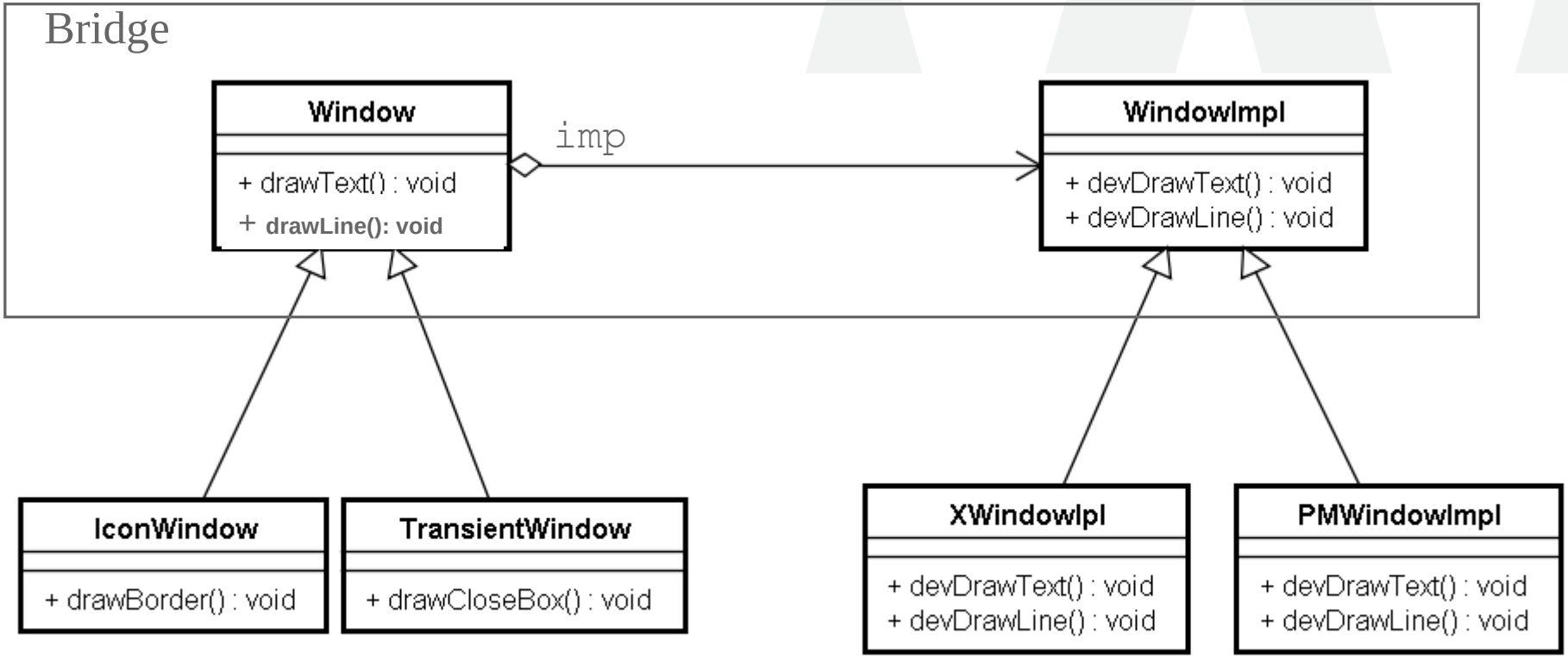
Bridge



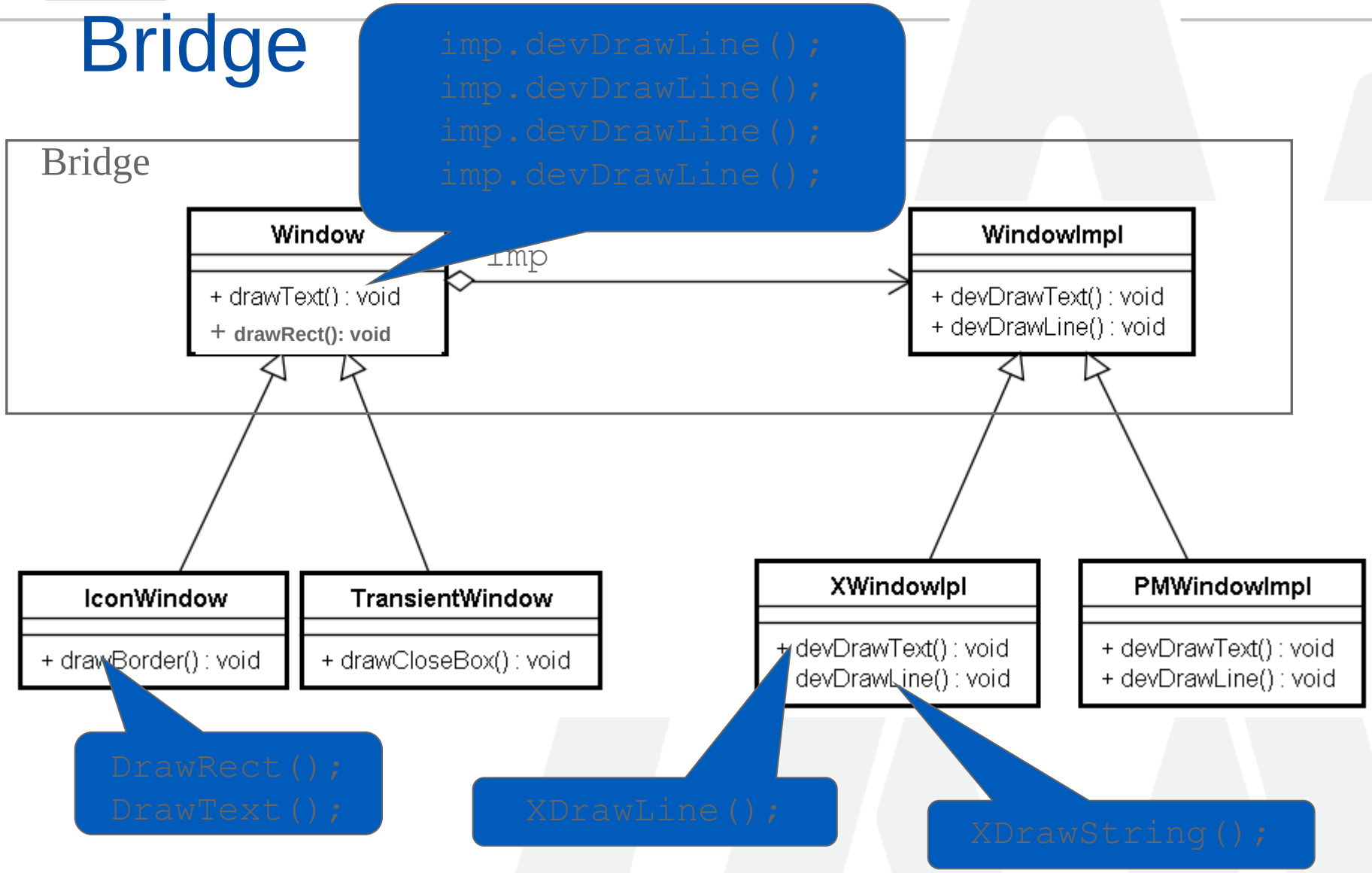


- Máme více hledisek klasifikace.
- Pokud bychom je chtěli zohlednit v jedné inheritanční hierarchii, došlo by k explozi počtu tříd (pro různé kombinace).
- Řešení – oddělit klasifikační hlediska.

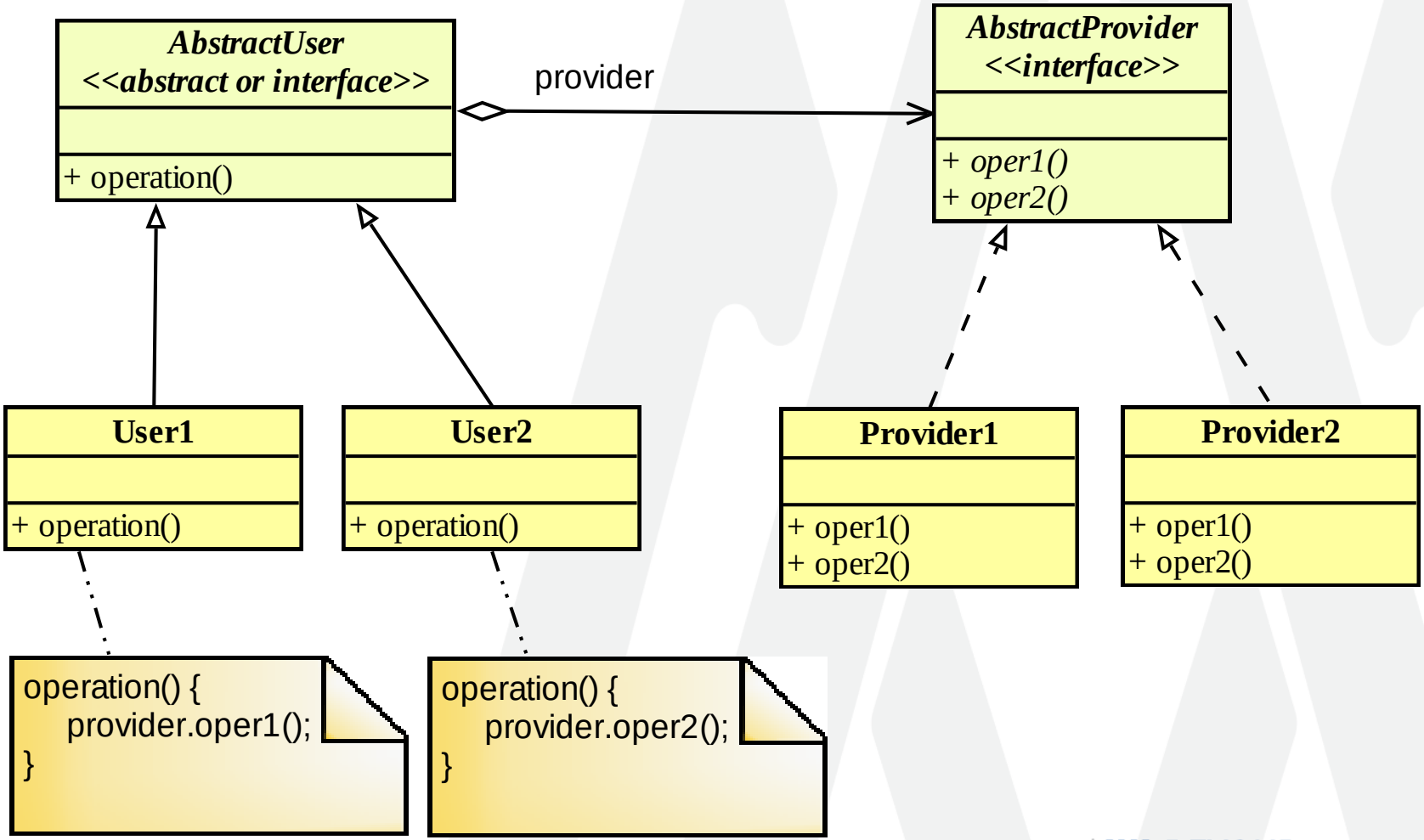
Bridge


Bridge
Window
+ drawText() : void
+ drawLine(): void
imp
WindowImpl
+ devDrawText() : void
+ devDrawLine() : void
IconWindow
+ drawBorder() : void
TransientWindow
+ drawCloseBox() : void
XWindowIpl
+ devDrawText() : void
+ devDrawLine() : void
PMWindowImpl
+ devDrawText() : void
+ devDrawLine() : void

Bridge


imp.devDrawLine();
imp.devDrawLine();
imp.devDrawLine();
imp.devDrawLine();
Bridge
Window
+ drawText() : void
+ drawRect(): void
imp
WindowImpl
+ devDrawText() : void
+ devDrawLine() : void
IconWindow
+ drawBorder() : void
TransientWindow
+ drawCloseBox() : void
XWindowIpl
+ devDrawText() : void
devDrawLine() : void
PMWindowImpl
+ devDrawText() : void
+ devDrawLine() : void
DrawRect();
DrawText();
XDrawLine();
XDrawString();

Bridge


AbstractUser
<<abstract or interface>>
+ operation()
provider
AbstractProvider
<<interface>>
+ oper1()
+ oper2()
User1
+ operation()
User2
+ operation()
Provider1
+ oper1()
+ oper2()
Provider2
+ oper1()
+ oper2()
operation() {
provider.oper1();
}
operation() {
provider.oper2();
}

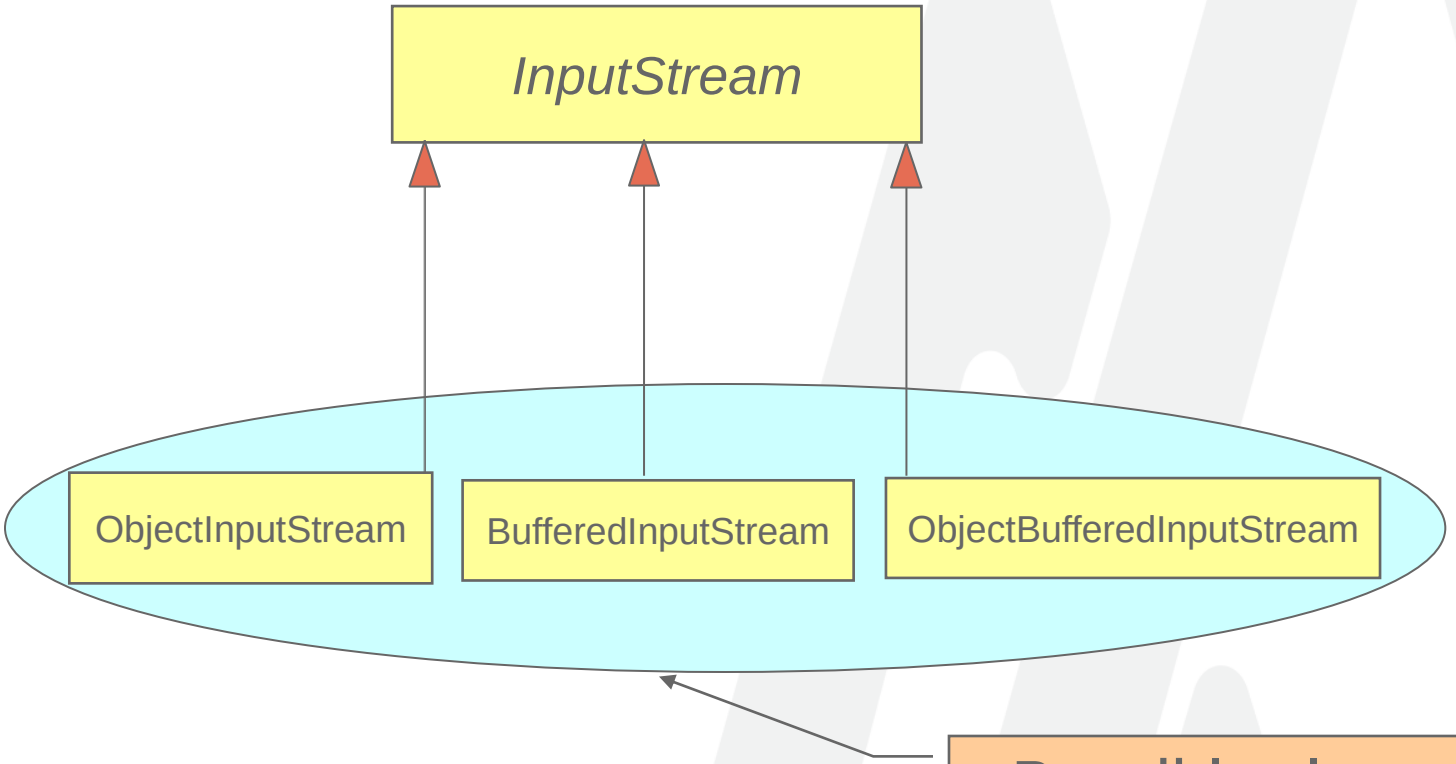
Bridge

1. The Bridge pattern is intended to keep the interface to your client class constant while allowing you to change the actual kind of class you use.

2. You can extend the implementation class and the bridge class separately, and usually without much interaction with each other (avoiding permanent binding between abstraction and its implementation).

3. You can hide implementation details from the client class much more easily.

Decorator



Possible class explosion
(hard to maintain)
Decorator patterns helps ...

The Open-Close Design Principle

Software entities like classes, modules and functions should be open for extension but closed for modifications.

Adding new functionality should involve minimal changes to existing code.

=> adding new functionality will not break the old functionality

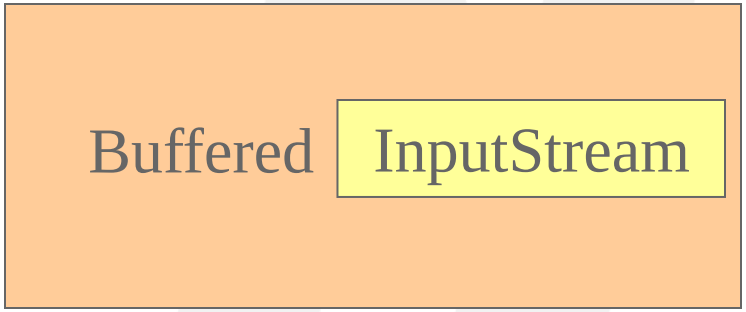
=> old functionality need not be re-tested

Most changes will be handled as new methods and new classes. Designs following this principle would result in resilient code which does not break on addition of new functionality.

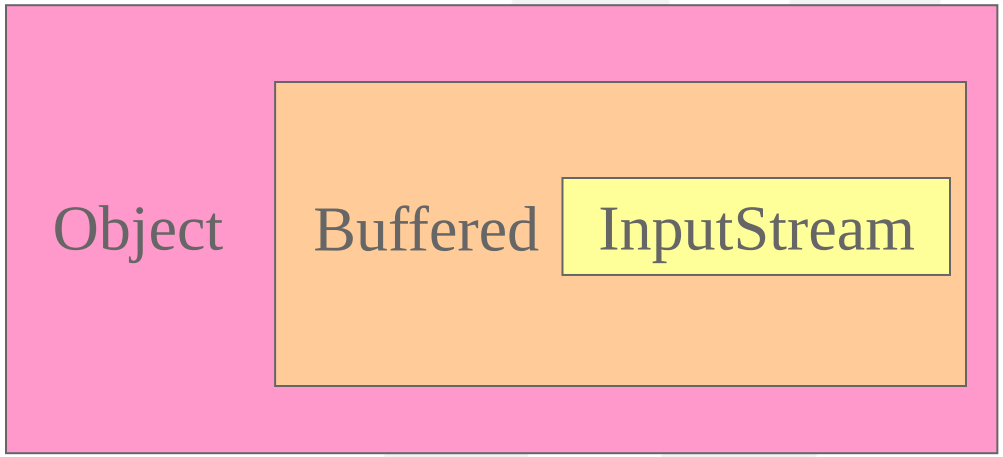
Decorator (structural)

InputStream

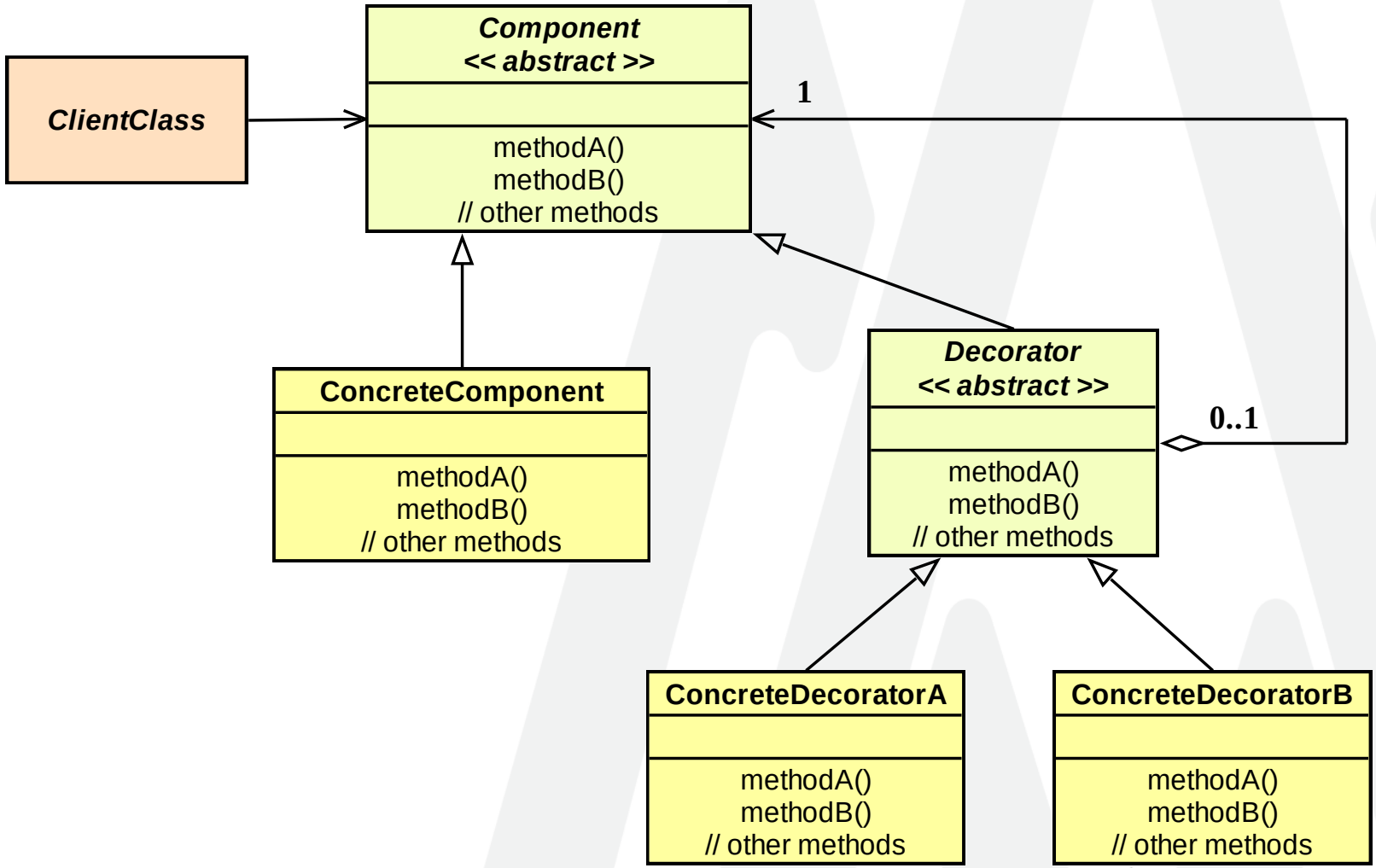
Decorator (structural)



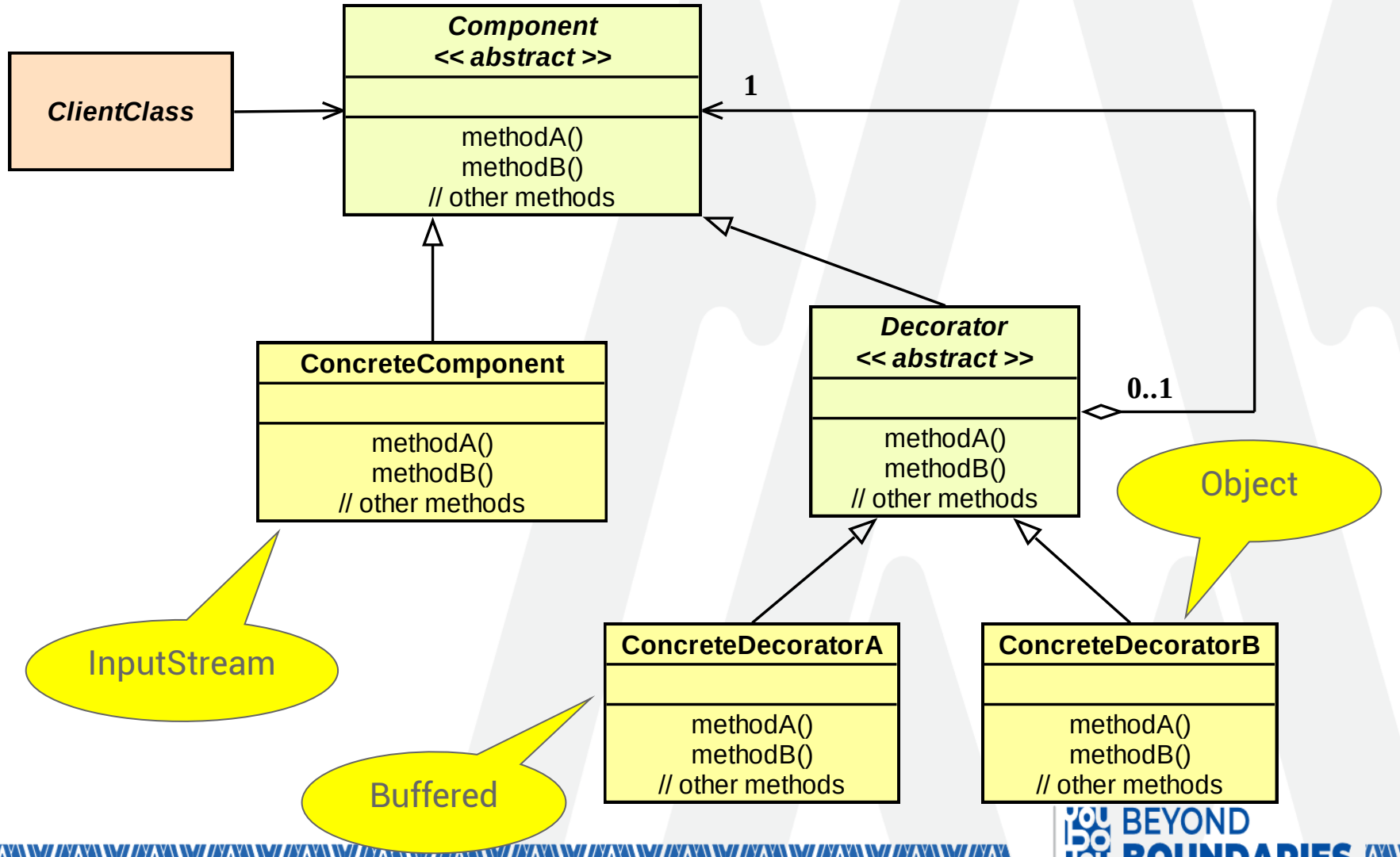
Decorator (structural)



Decorator (structural)



Decorator (structural)



Decorator (structural)

```
InputStream is = new FileInputStream ("fff");
BufferedInputStream bis = new BufferedInputStream(is);

ObjectInputStream ois = new ObjectInputStream(bis);
```

```
BufferedInputStream bis = new BufferedInputStream(new FileInputStream("fff"));
```

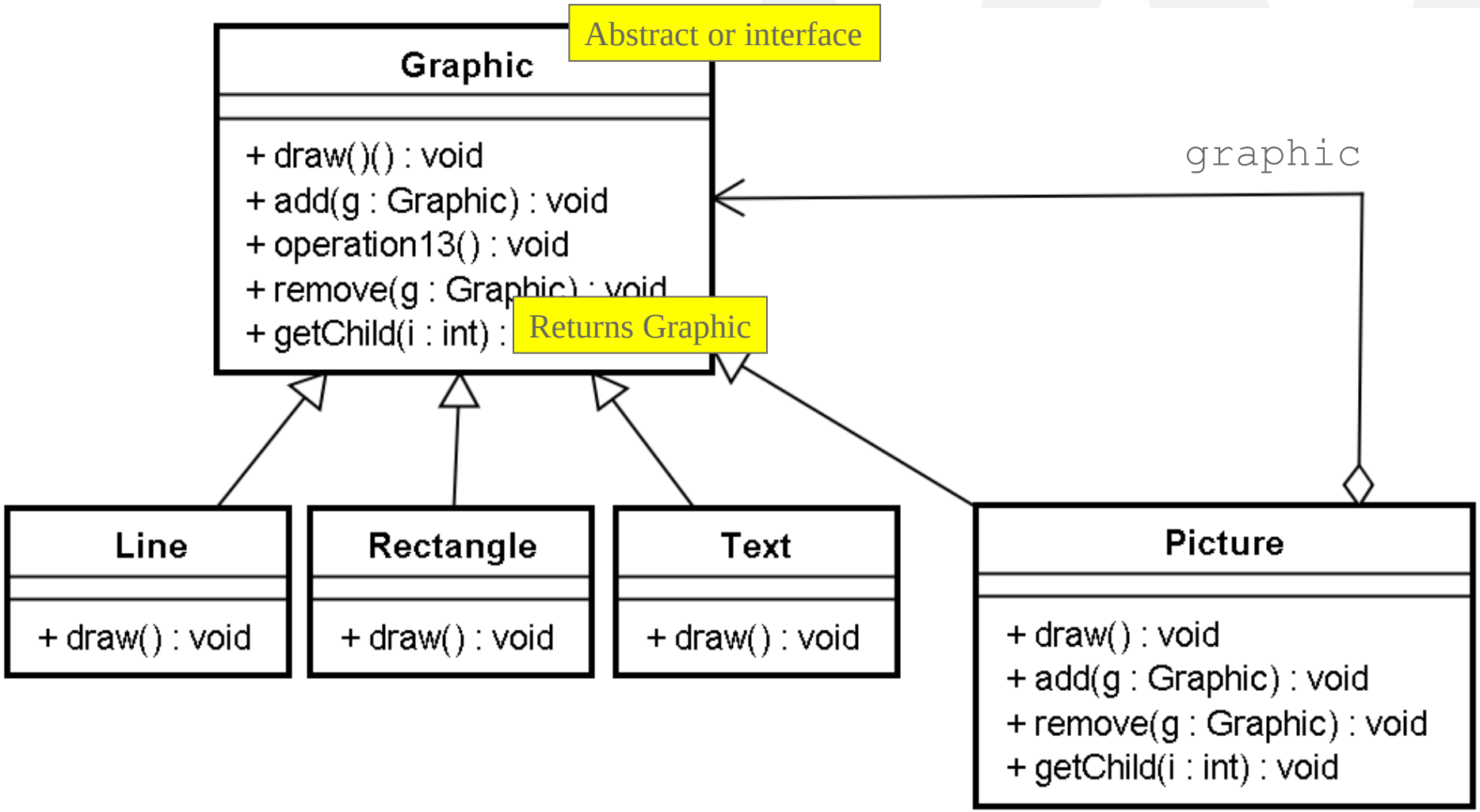
Decorator (structural)

The Decorator Pattern attaches additional functionalities to an object dynamically.

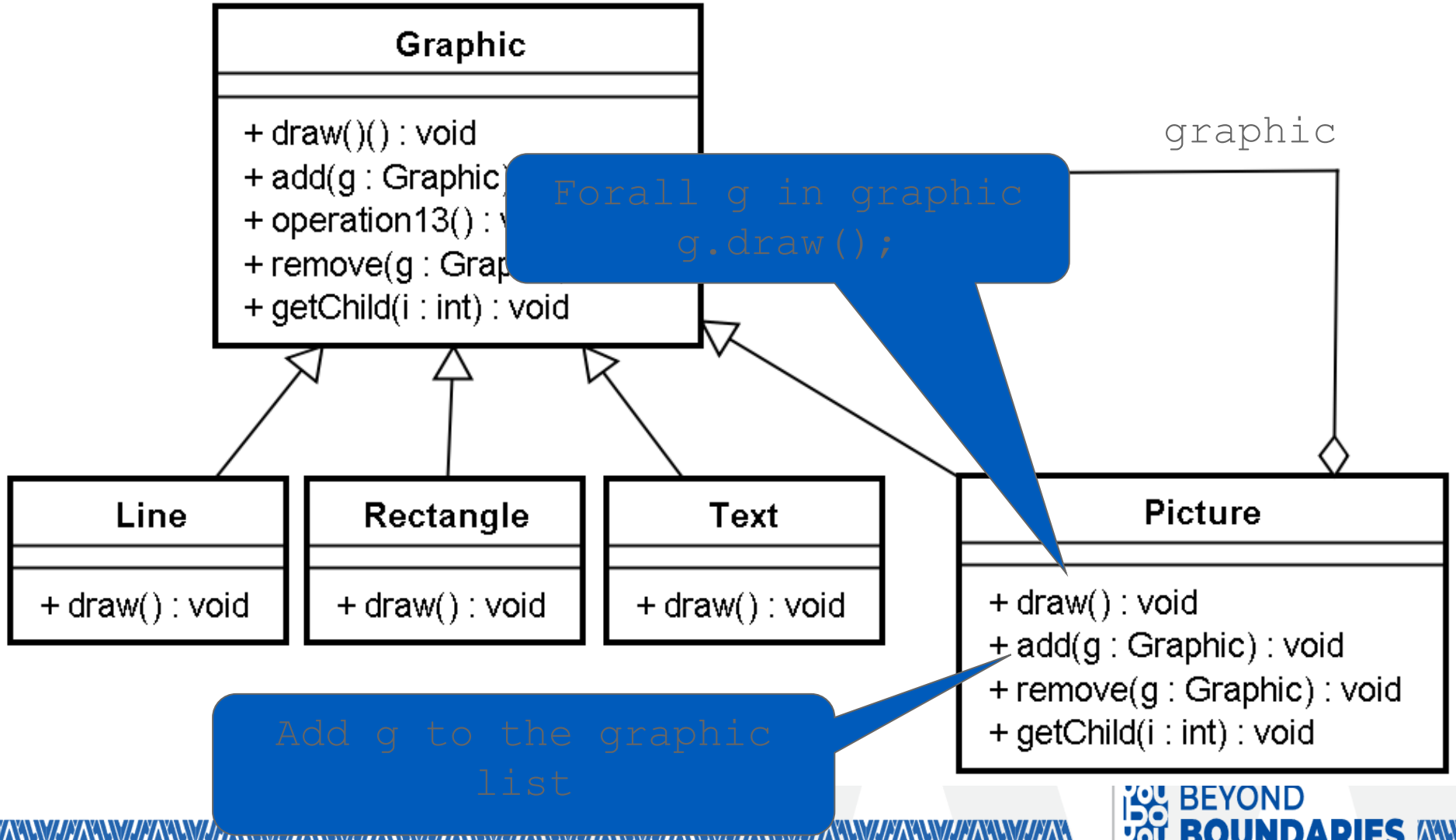
Decorators provide a flexible alternative to subclassing for extending functionality.

Decorator prevents explosion of (sub)classes.

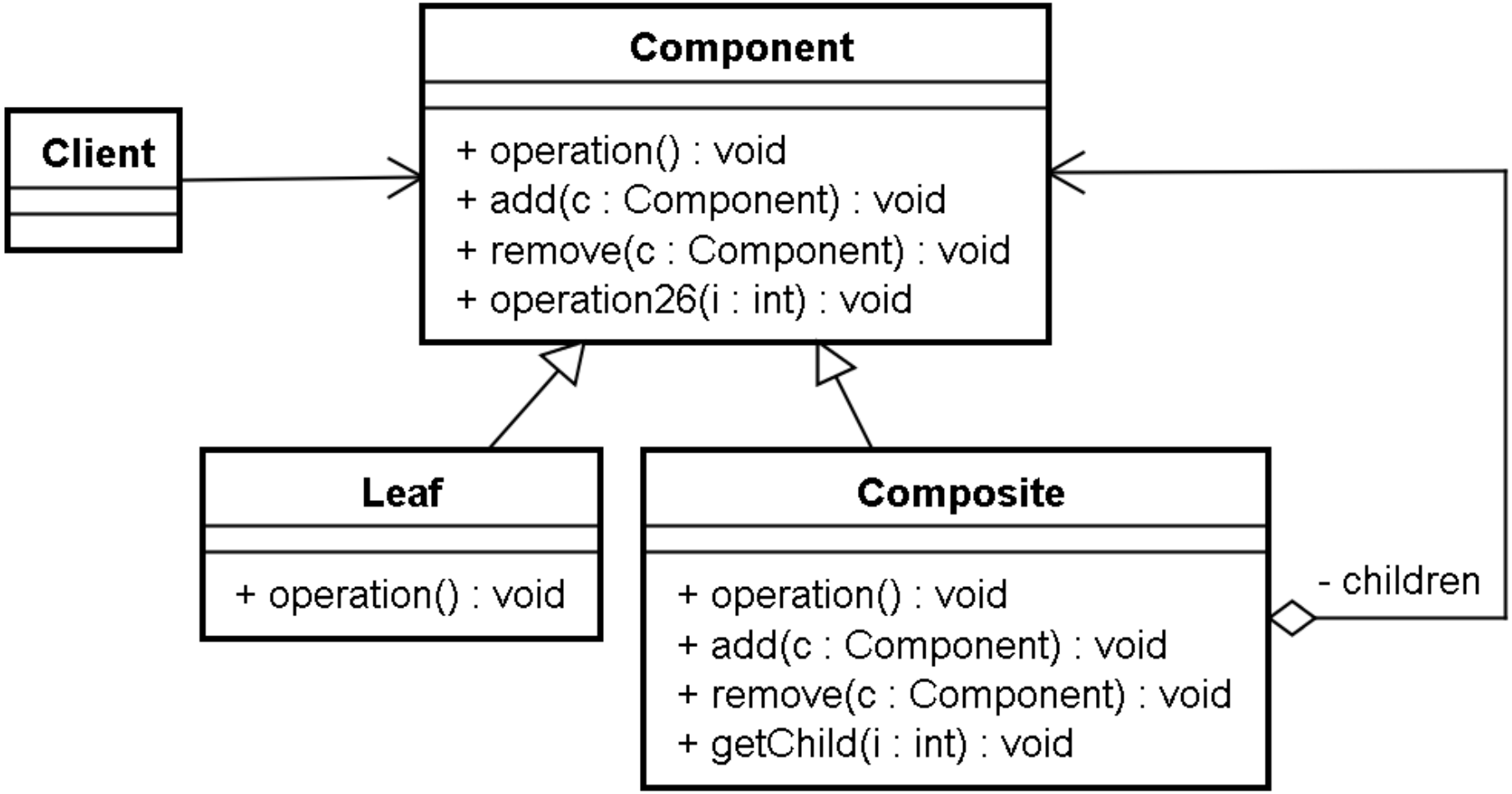
Composite (structural)



Composite (structural)

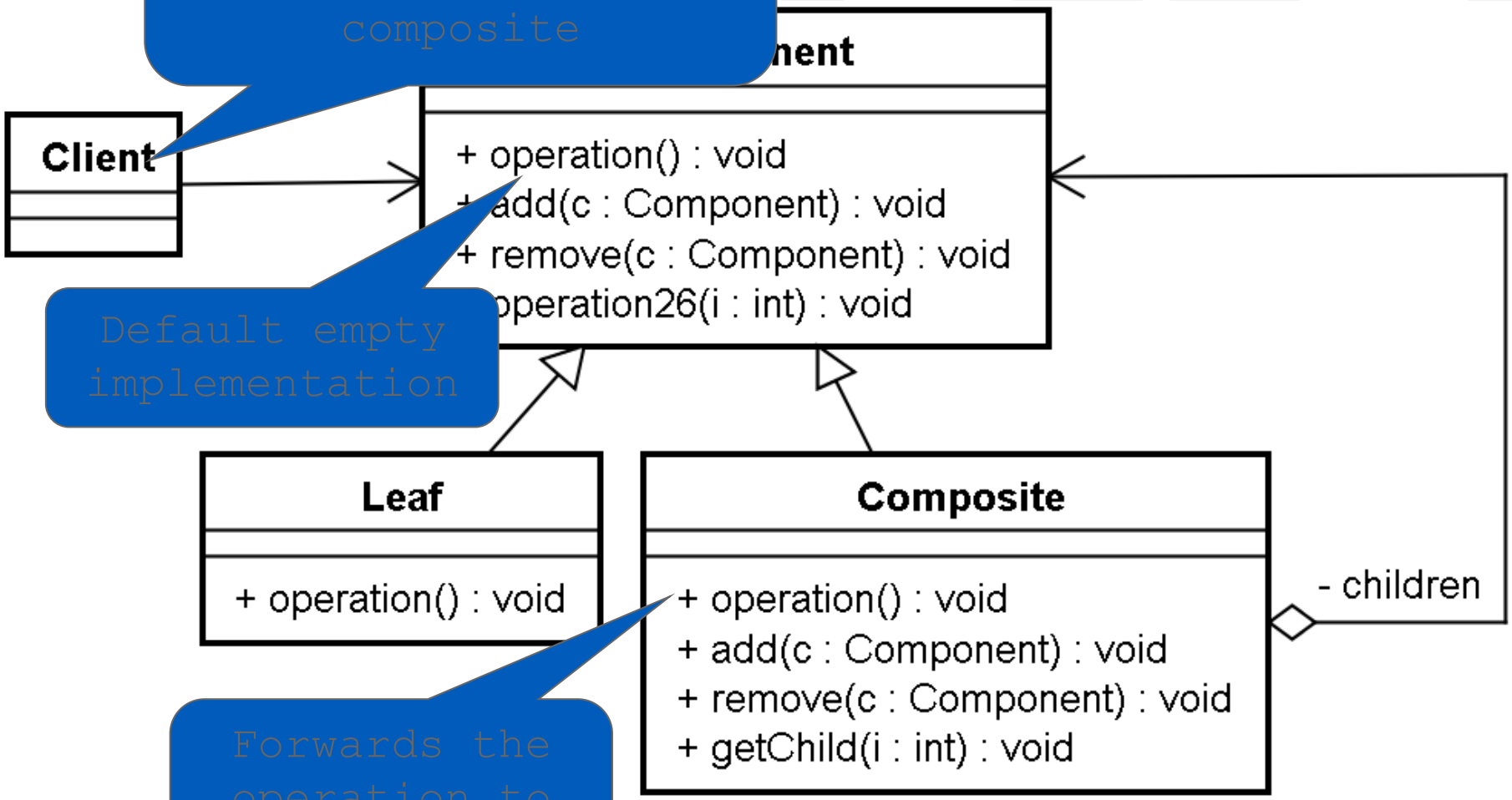


Composite (structural)



Co

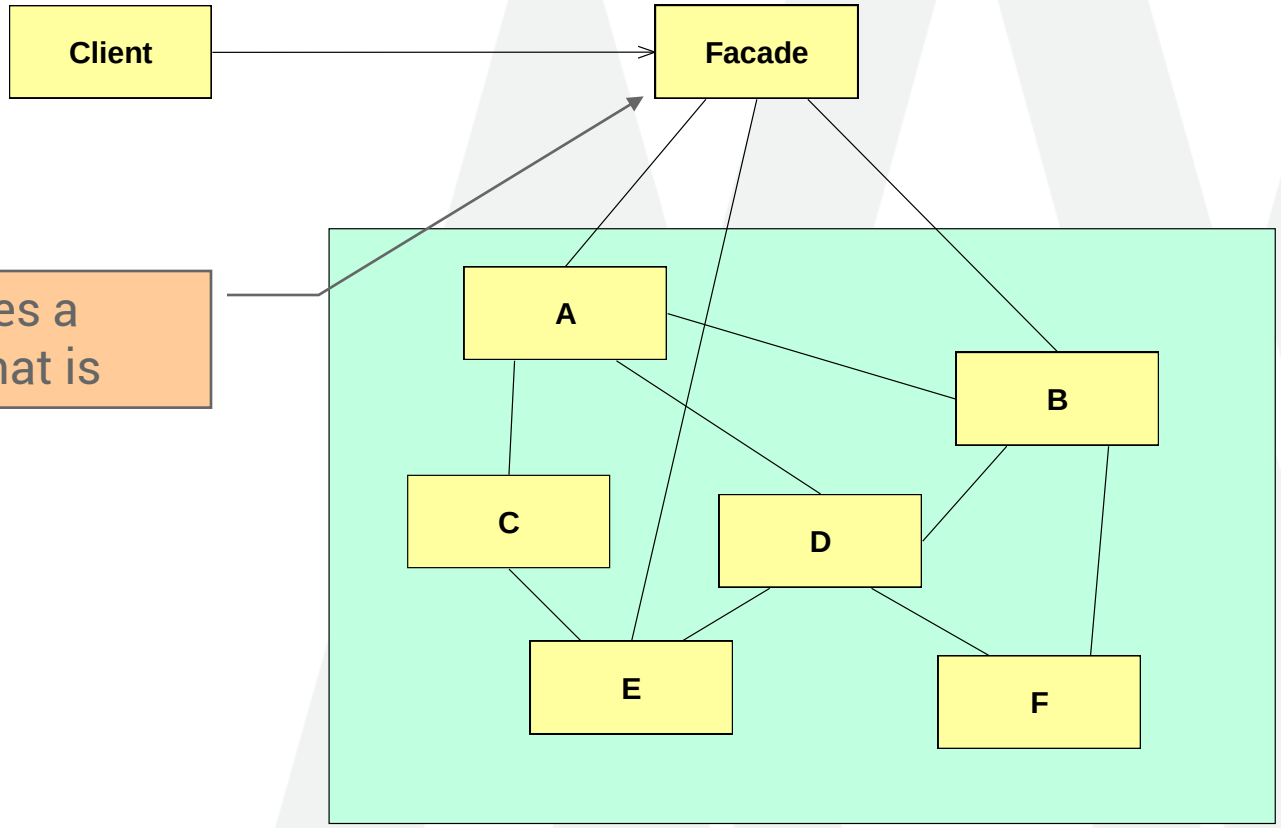
Views all components uniformly regardless whether leaf or composite



Default empty implementation

Forwards the operation to all children

Façade (structural)



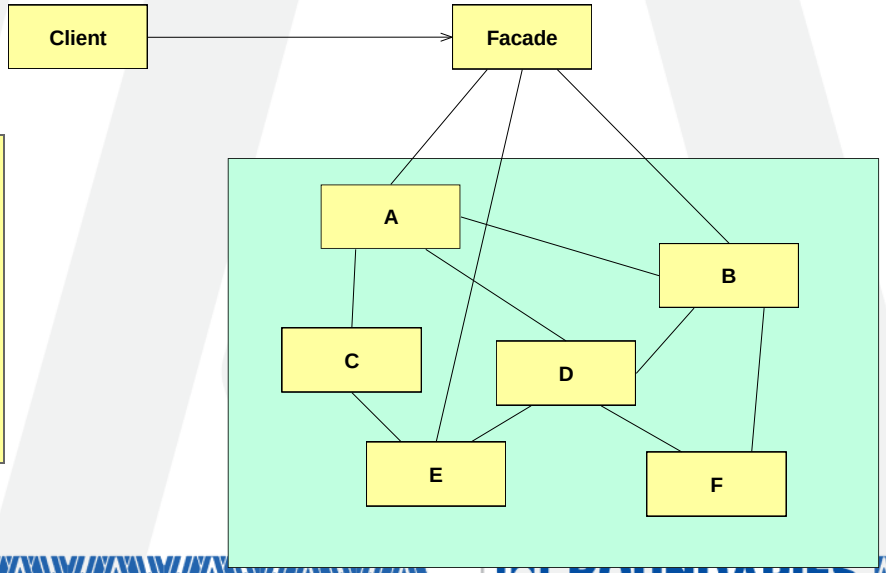
The façade provides a unified interface that is easier to use

A complex subsystem

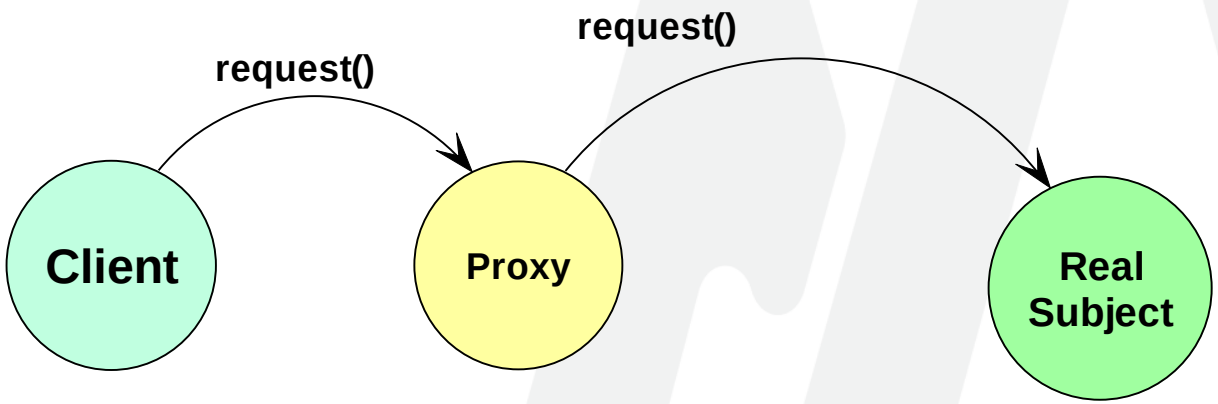
Façade (structural)

The Façade Pattern provides a unified interface to a set of interfaces in a subsystem. Façade defines a higher-level interface that makes the system easier to use.

Design Principle
Principle of least knowledge - talk only to your immediate friends.

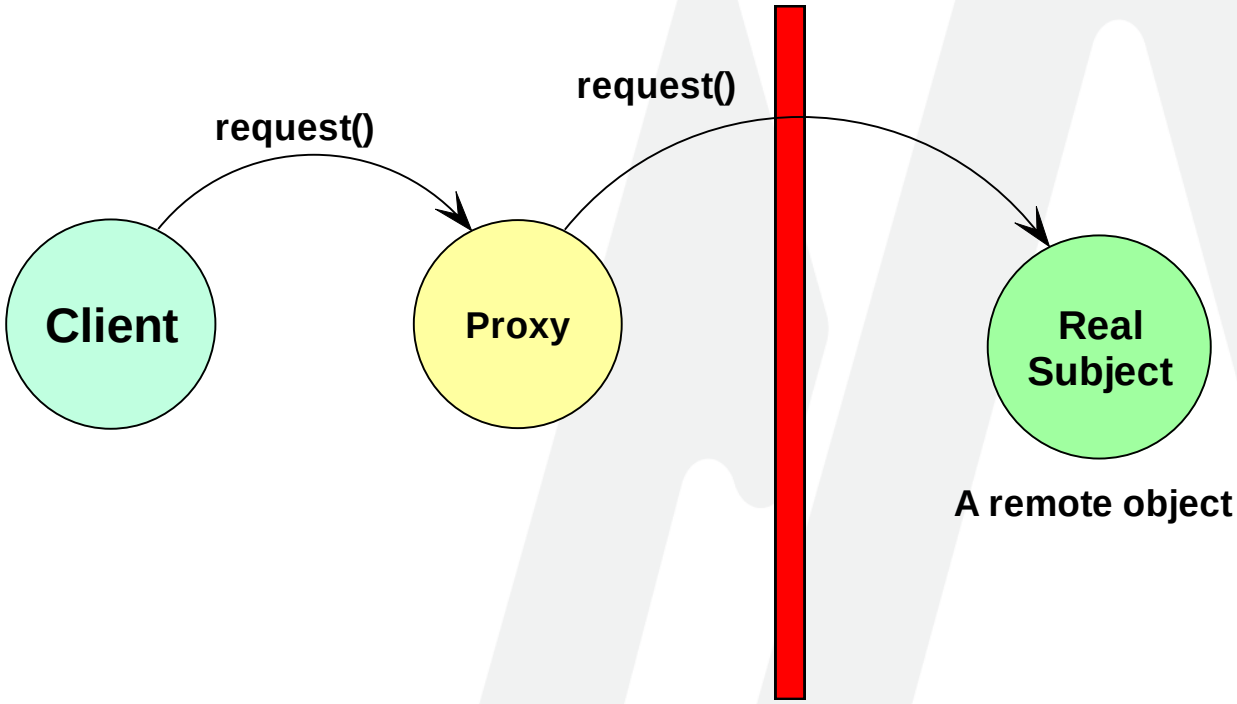


Proxy (structural)



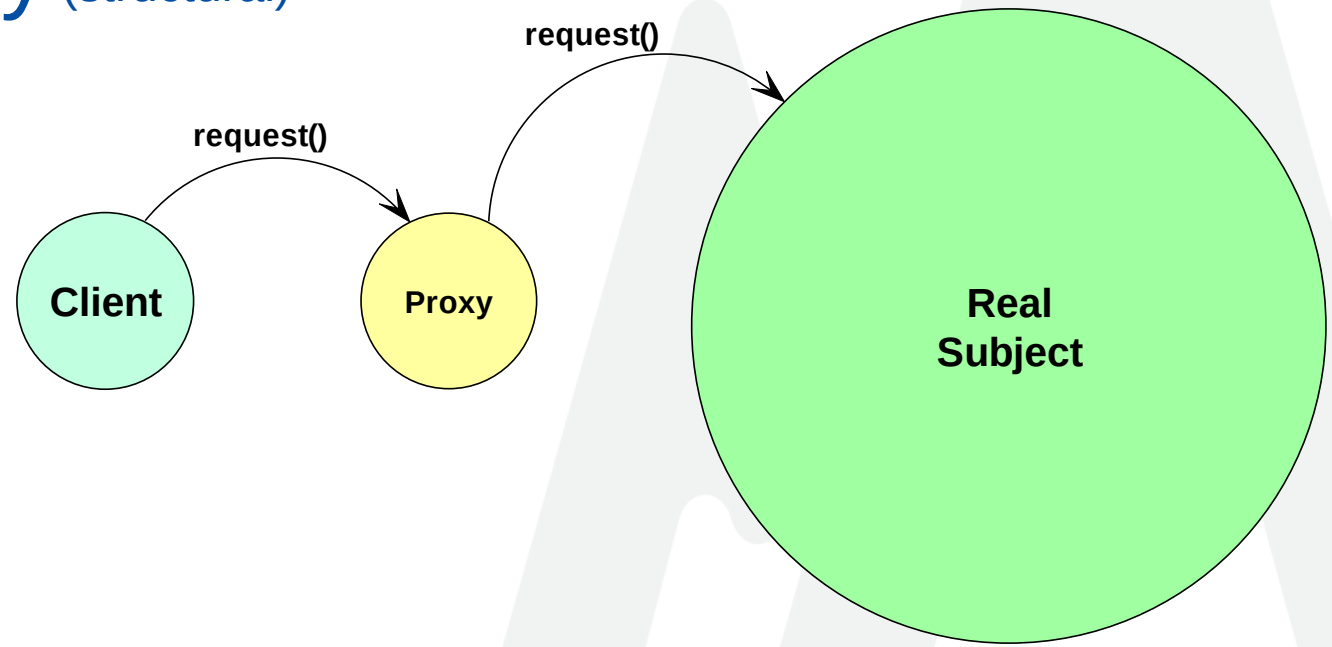
The Proxy Pattern provides a surrogate or placeholder for another object to access control to it.

Proxy (structural)



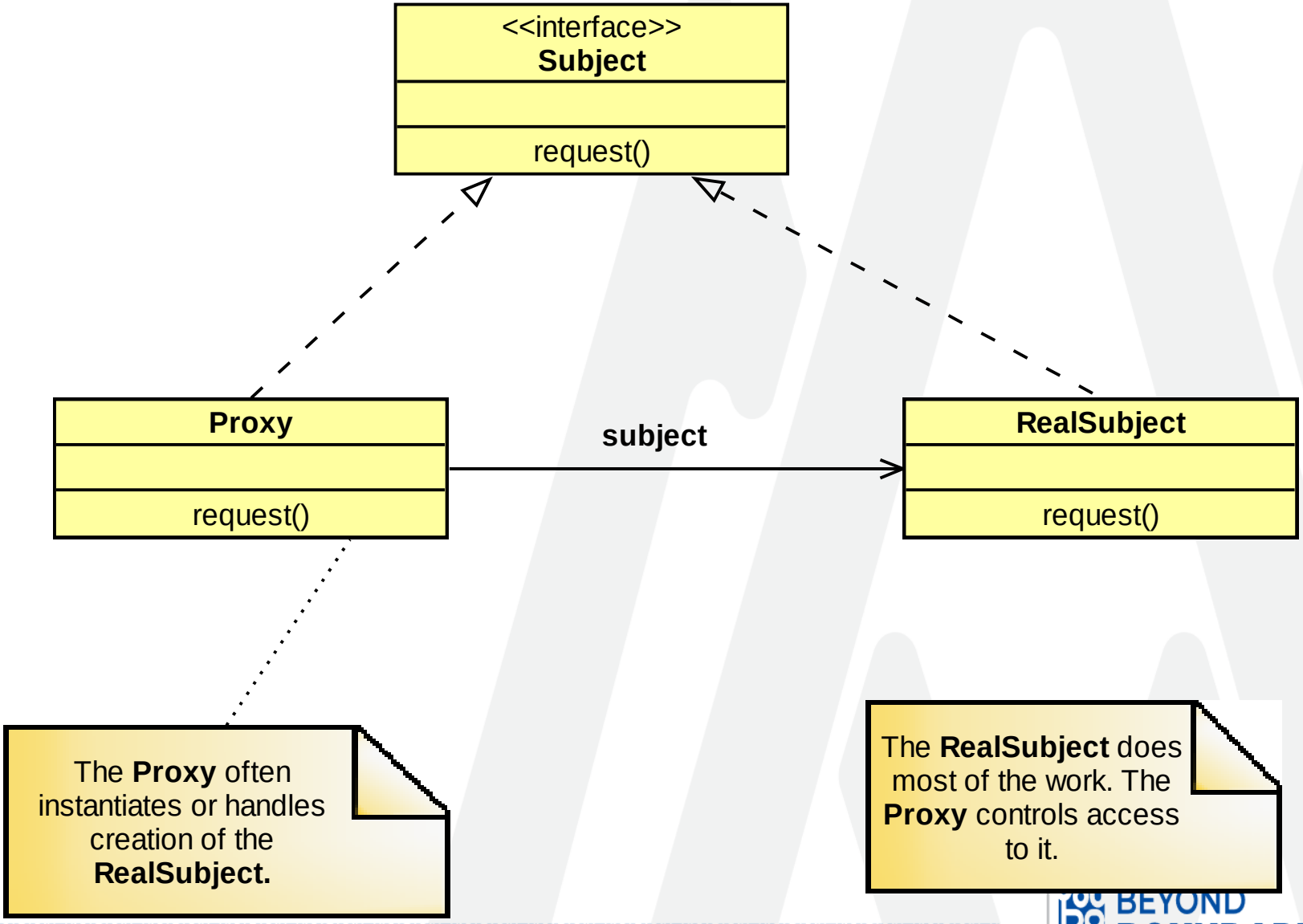
With **Remote Proxy**, the proxy act as a local representative for an object that lives in a different JVM.

Proxy (structural)



Virtual Proxy acts as a representative for an object that may be expensive to create. The virtual proxy often defers creation of the object until it is needed. After that, the virtual proxy delegates requests to the RealSubject.

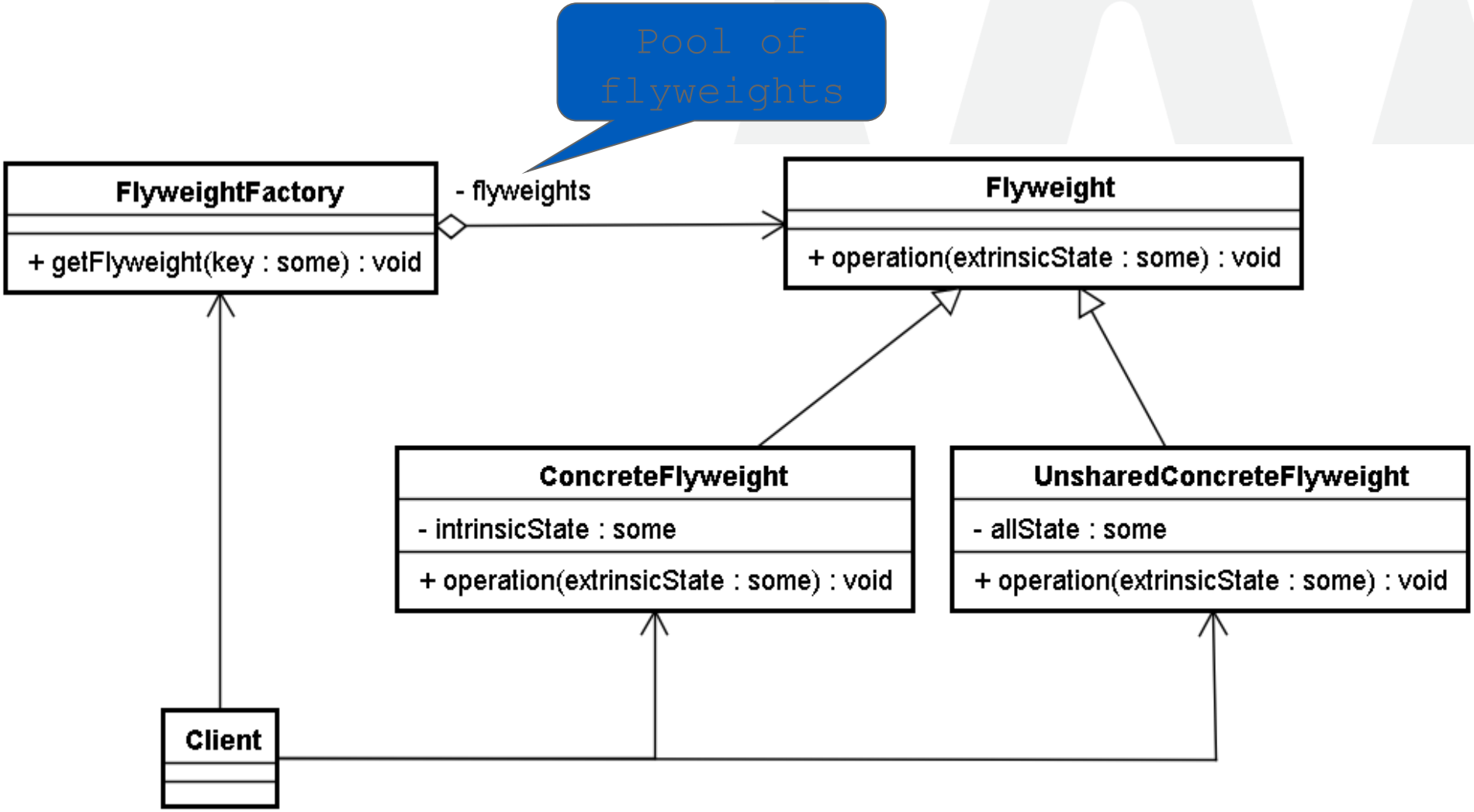
Proxy (structural)



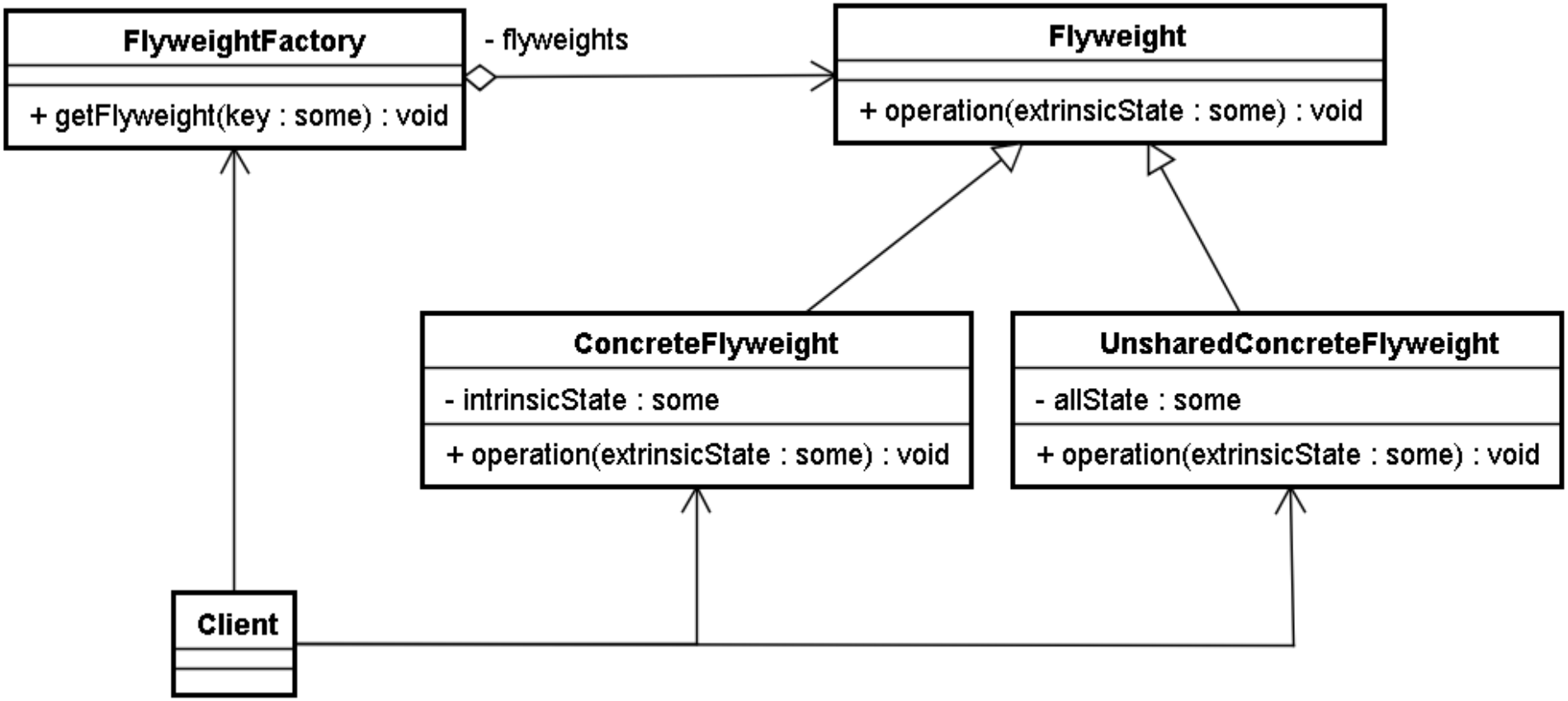
Flyweight (structural)

- Intrinsic state is stored in the `ConcreteFlyweight` object
- Extrinsic state is stored or computed by `Client` objects. Clients pass this state to the flyweight when they invoke they operations
- Clients should not instantiate `ConcreteFlyweights` directly. Clients must obtain `ConcreteFlyweights` from the `FlyweightFactory` to ensure they are shared properly.
- not all Flyweight objects need to be shared. It is common for `UnsharedConcreteFlyweight` objects to have `ConcreteFlyweight` objects as children.

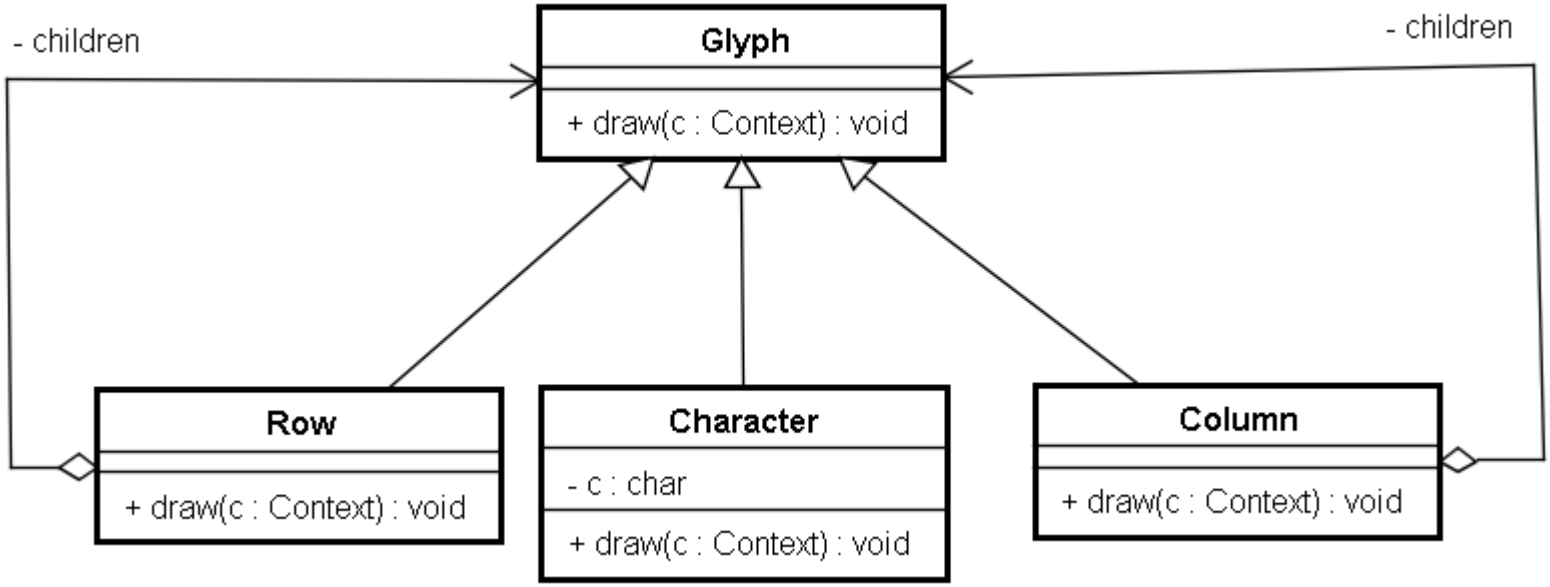
Flyweight (structural)



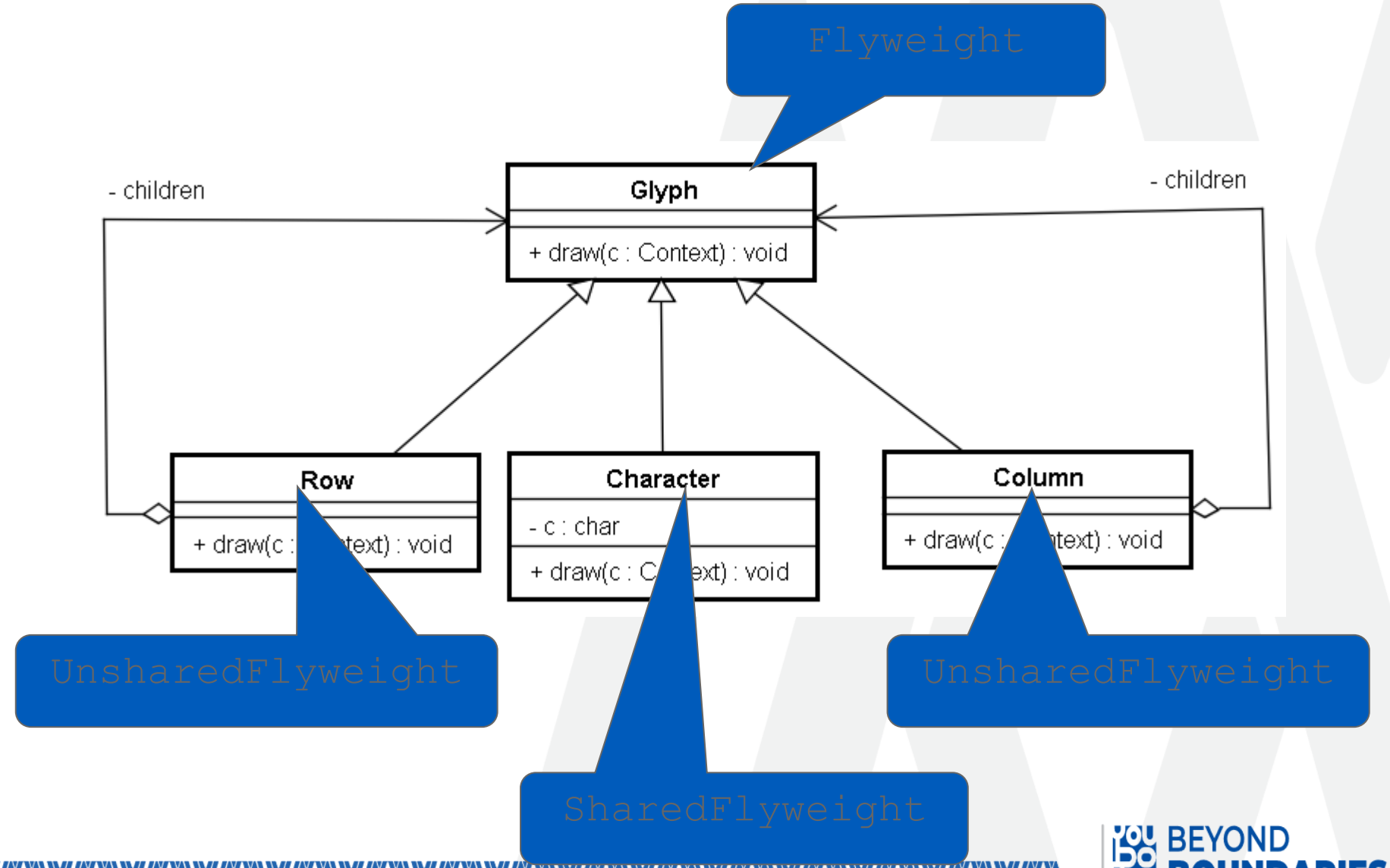
Flyweight (structural)



Flyweight (structural)



Flyweight (structural)



Behavioral Design Patterns

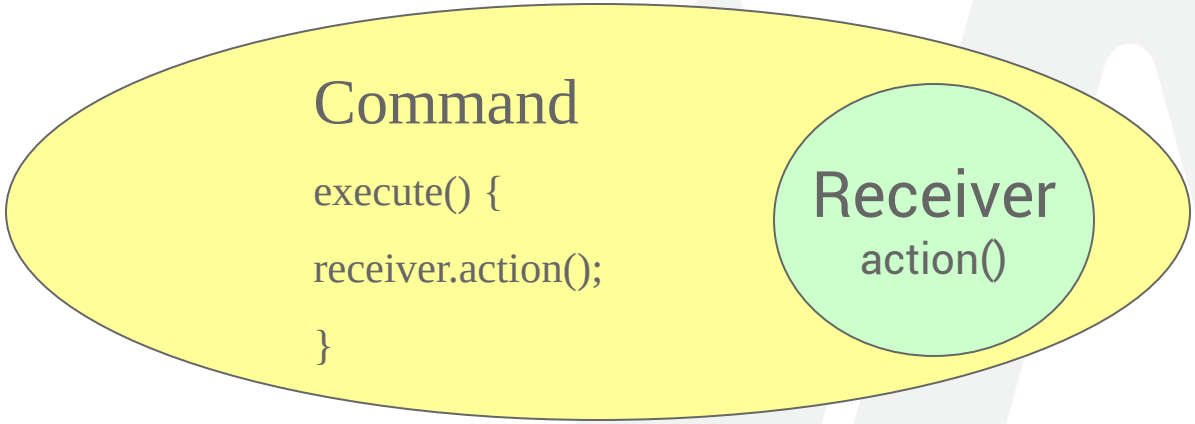
- Command
- Interpreter
- Iterator
- Mediator
- Observer
- State
- Strategy
- Visitor

Command (behavioral)

Command represents an abstract algorithm independent on:

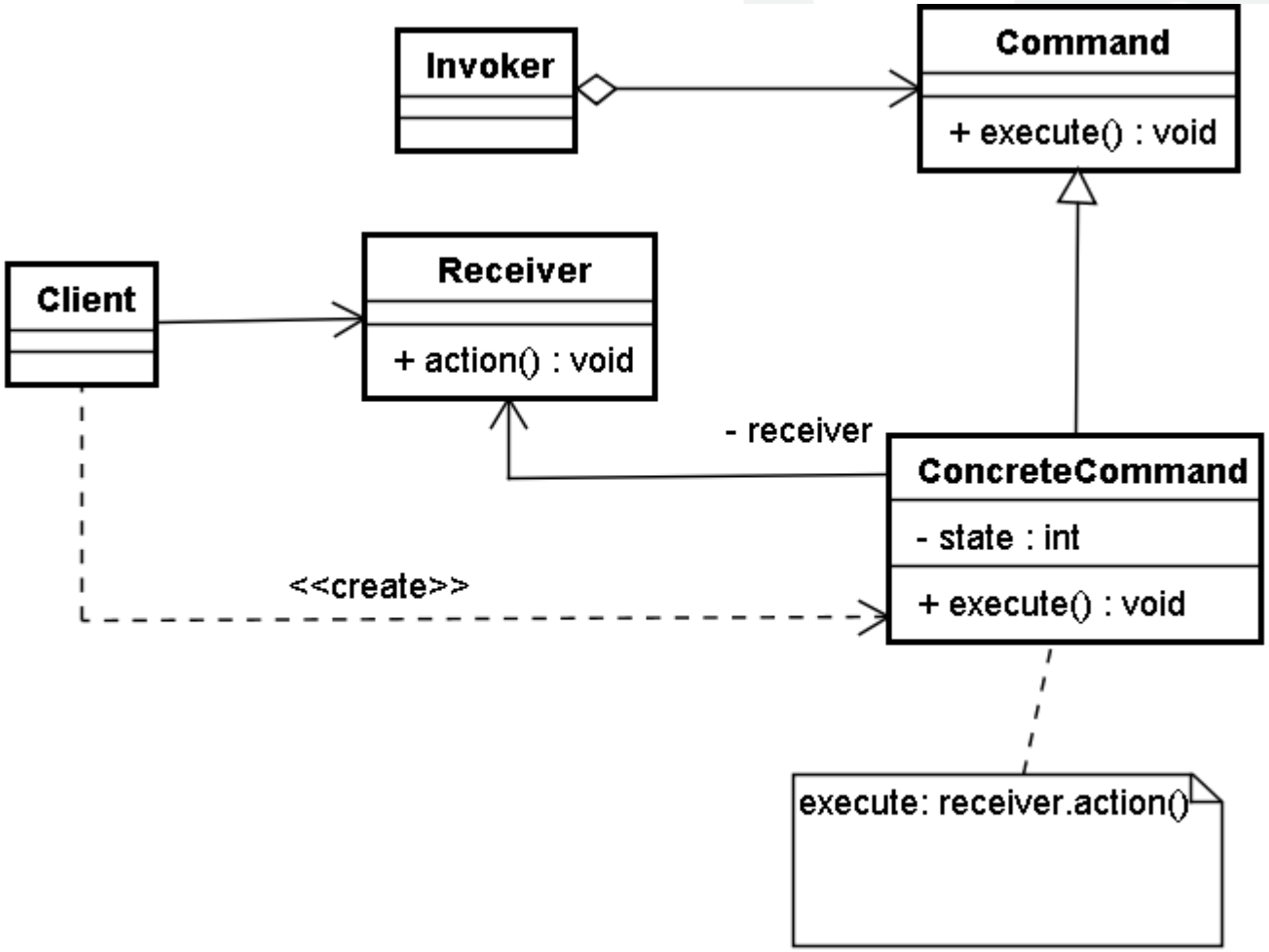
1) The application of the command (Client)
2) The particular implementation of the command (Receiver)

Command (behavioral)

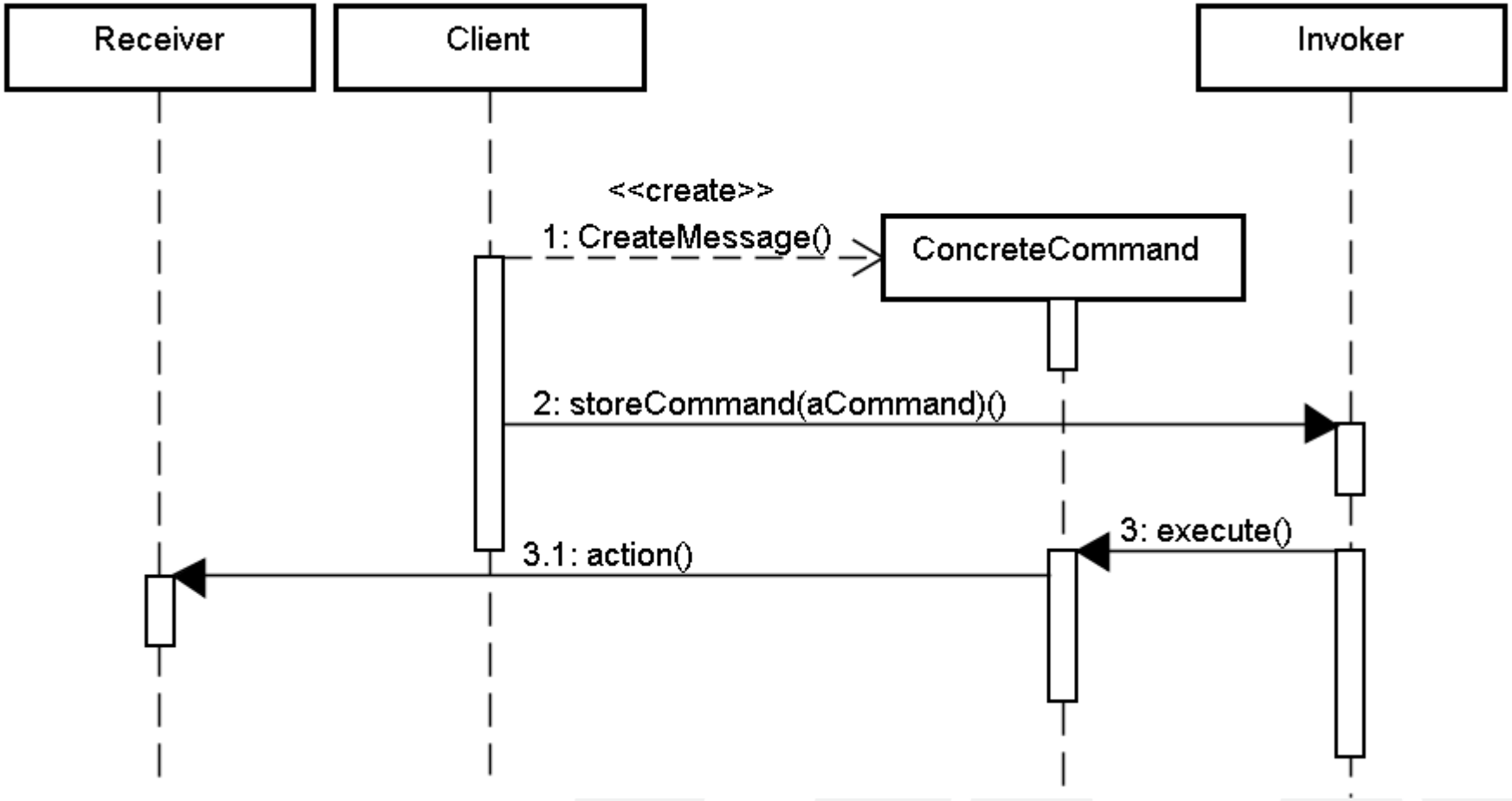


Command encapsulates a **request** as an object, thereby letting you parametrize other objects with different requests, queue or log requests and support undoable operations.

Command (behavioral)



Command (behavioral)



Command (behavioral)

- Command decouples the object that invokes the operation from the one that knows how to perform it.
- You can assemble commands into a composite command (composite commands are instances of Composite pattern)
- Easy to add new commands.

Behavioral Design Patterns

Behavioral **Class** Patterns

Use inheritance to distribute behaviour between classes

- Template Method
-

Behavioral **Object** Patterns

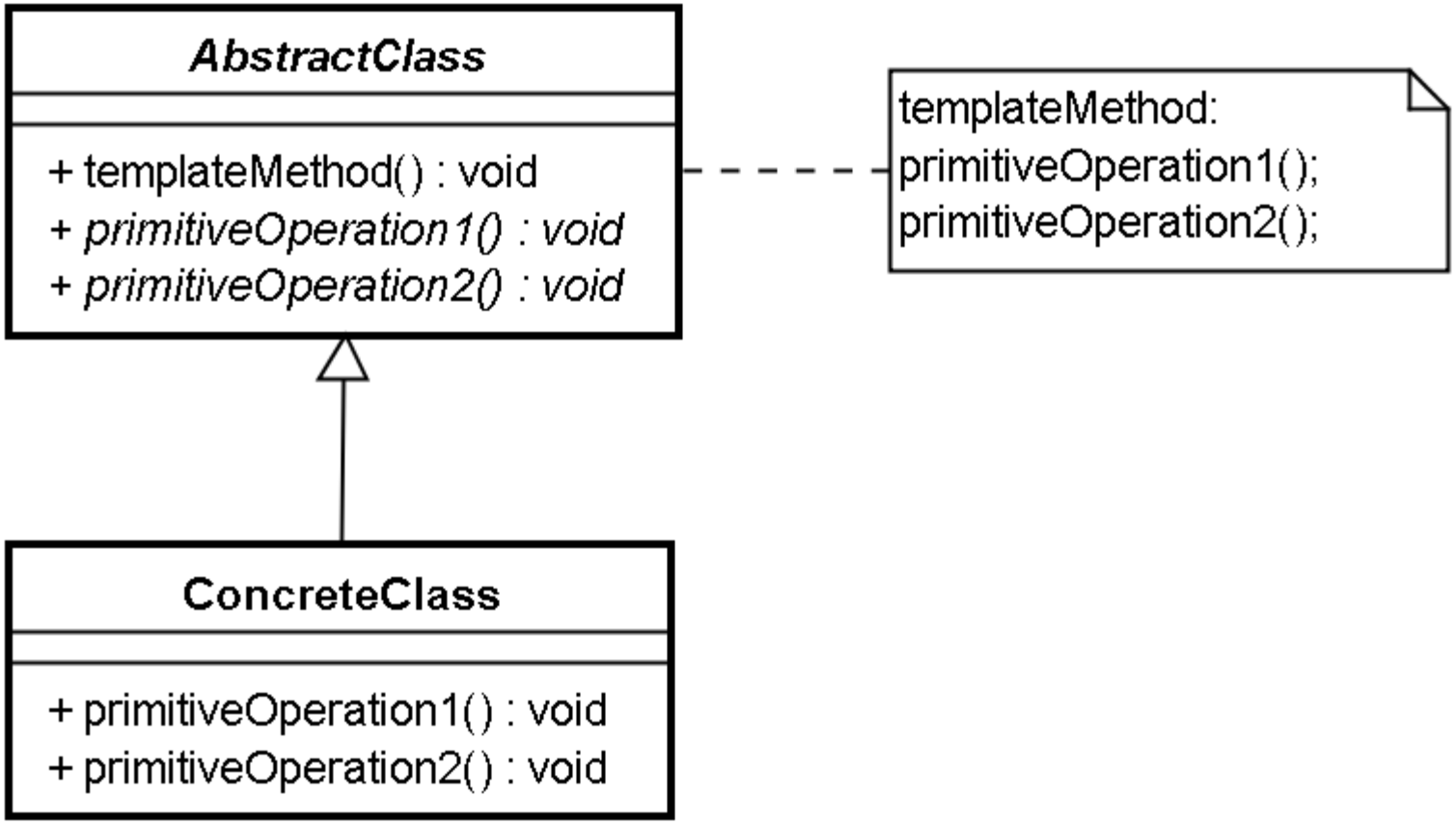
Use composition rather than inheritance to distribute behaviour between objects

- Mediator
-
- Observer

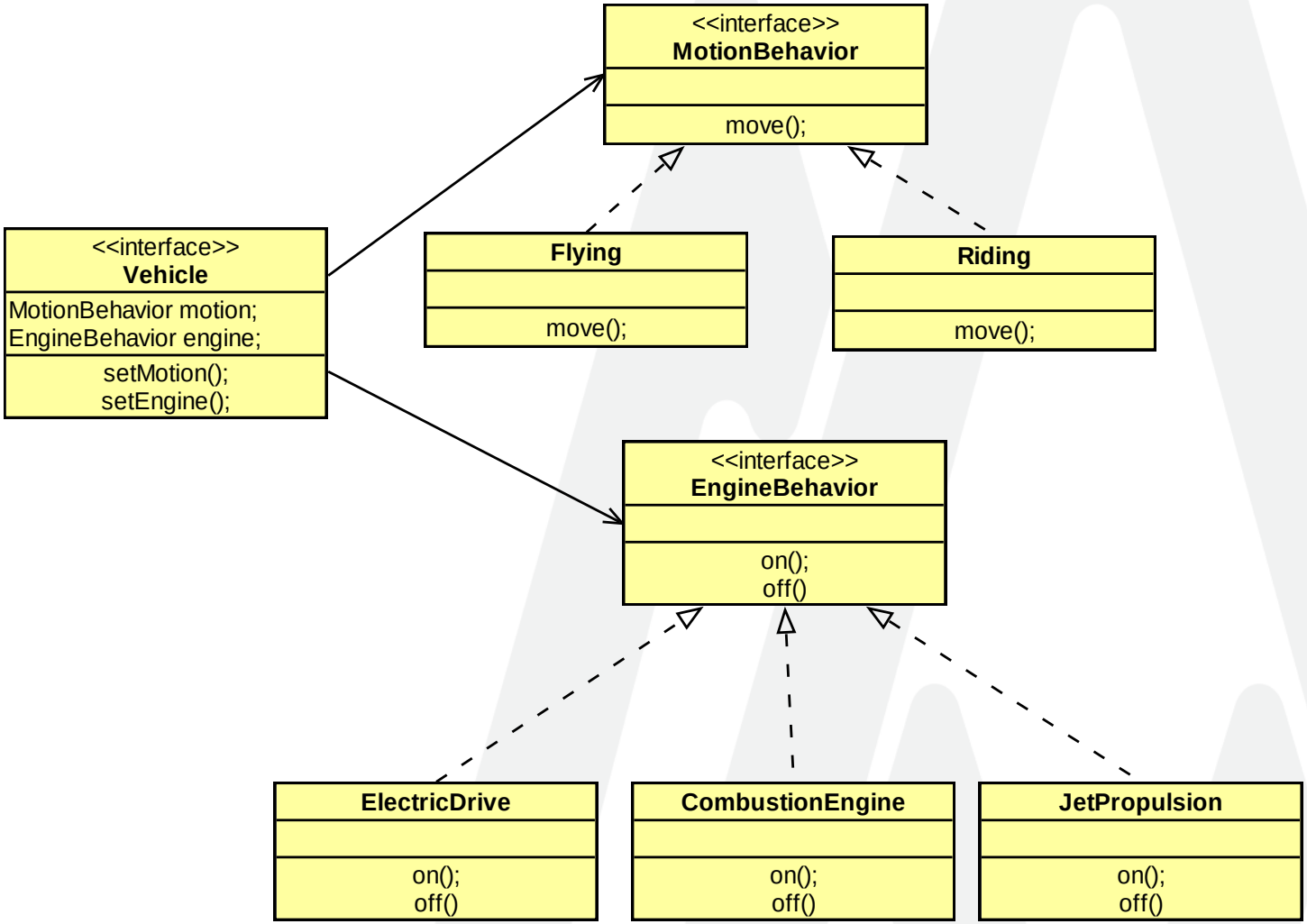
Other Behavioral Patterns

- Strategy
- Command
- Visitor
-

Template Method



Strategy (behavioral)



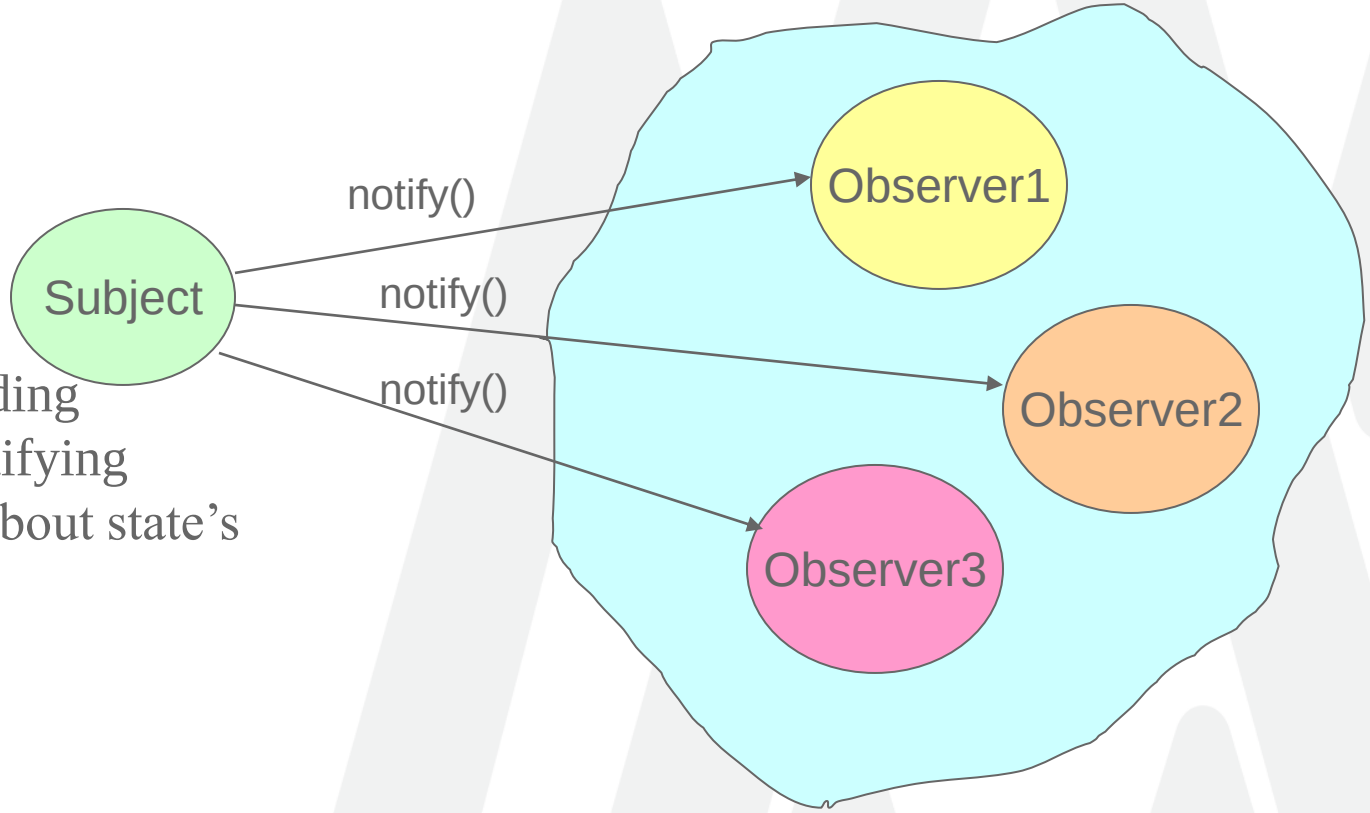
Strategy (behavioral)

The **Strategy Pattern** defines a family of algorithms, encapsulates each one, and makes them interchangeable. Startegy lets the algorithm vary independently from clients that use it.

Desing Principal

Favor composition over inheritance.

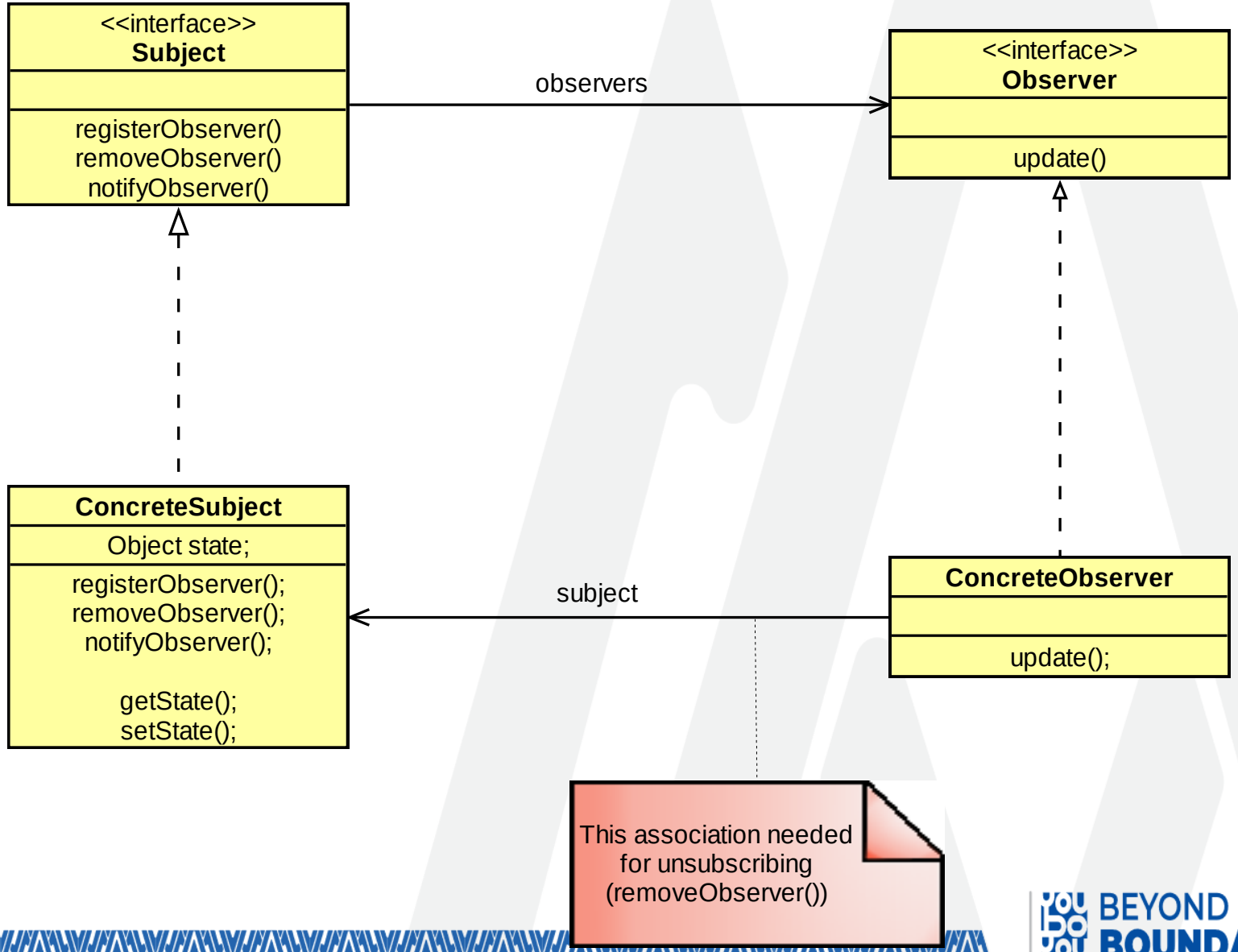
Observer (behavioral)



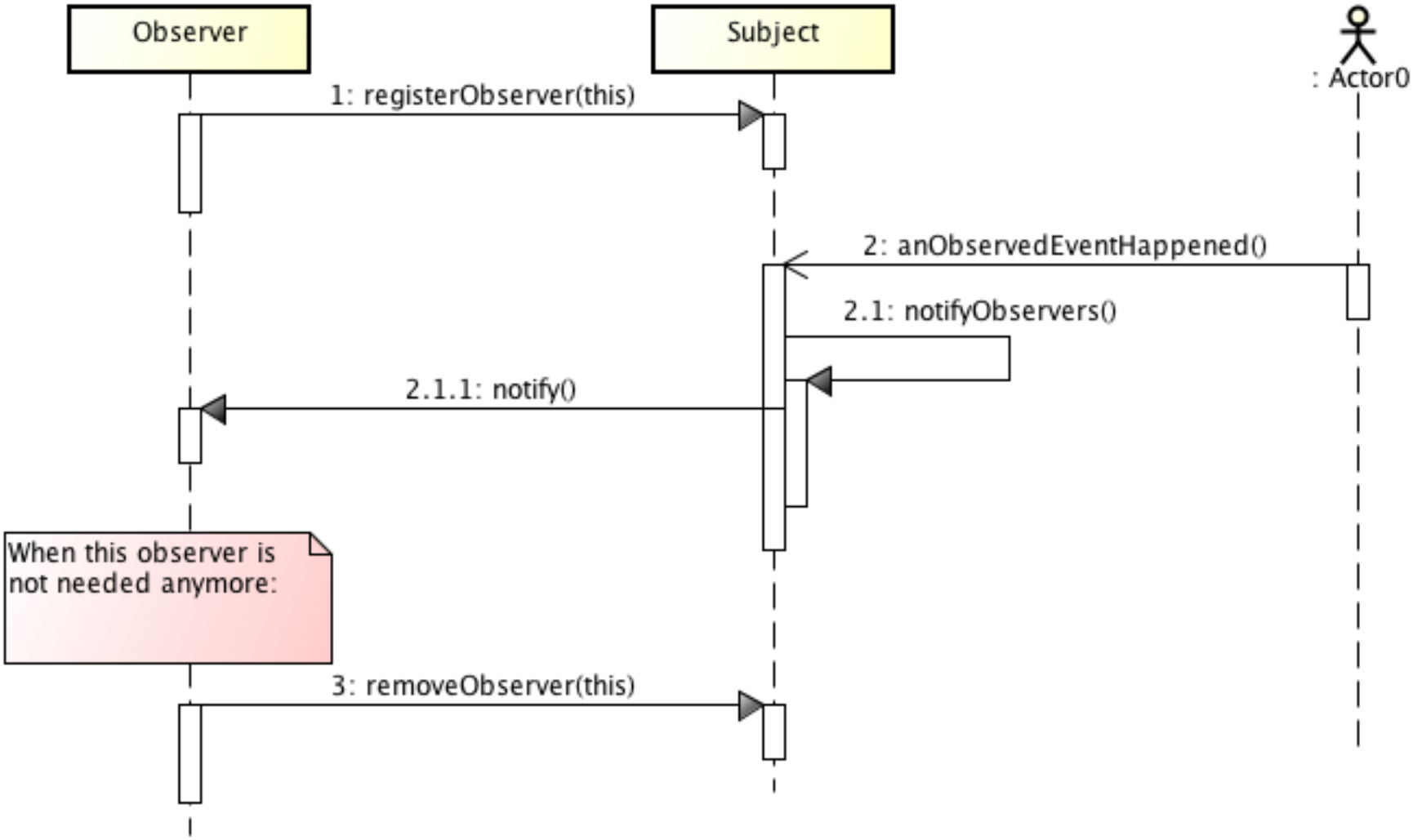
An object holding a state and notifying its observers about state's change.

The aim is to make subject independent on observers - loose coupling.

Observer (behavioral)



Observer (behavioral)



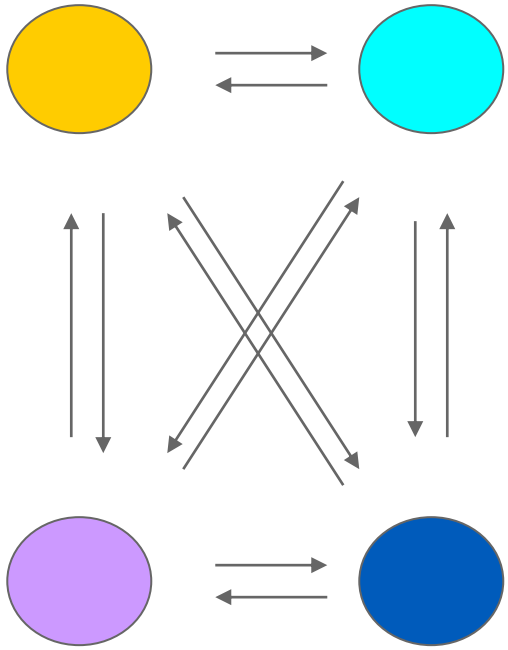
Observer (behavioral)

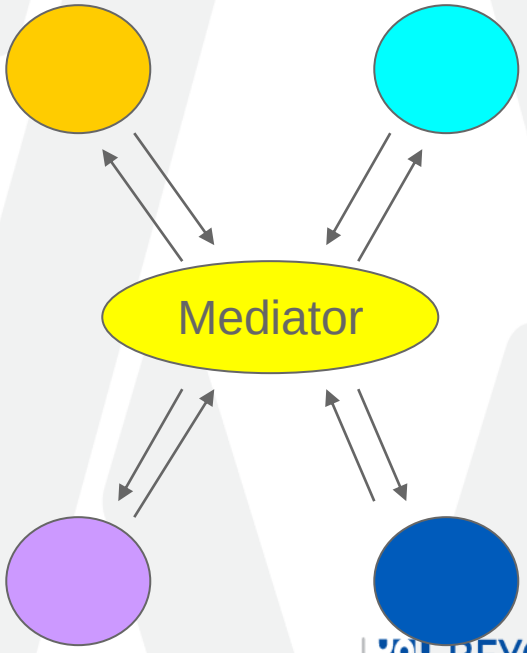
Desing Principle

Loosely coupled designs allows us to build flexible OO systems that can handle change because they minimize the interdependency between objects.

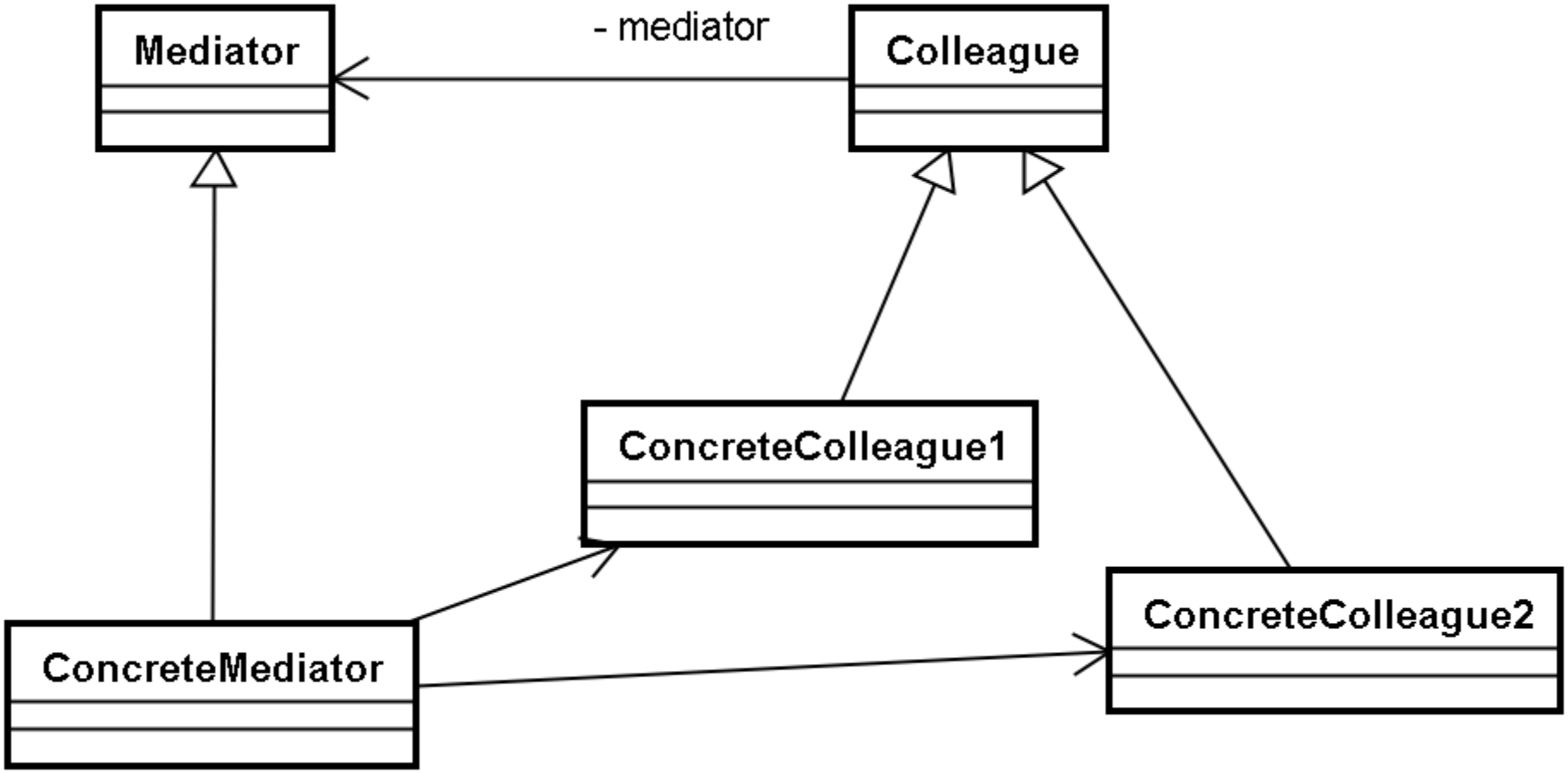
Mediator (behavioral)

- define an object that encapsulates how a set of objects interacts
- mediator promotes loose coupling by keeping objects from refering to each other explicitly
- it lets you vary their interactions independently

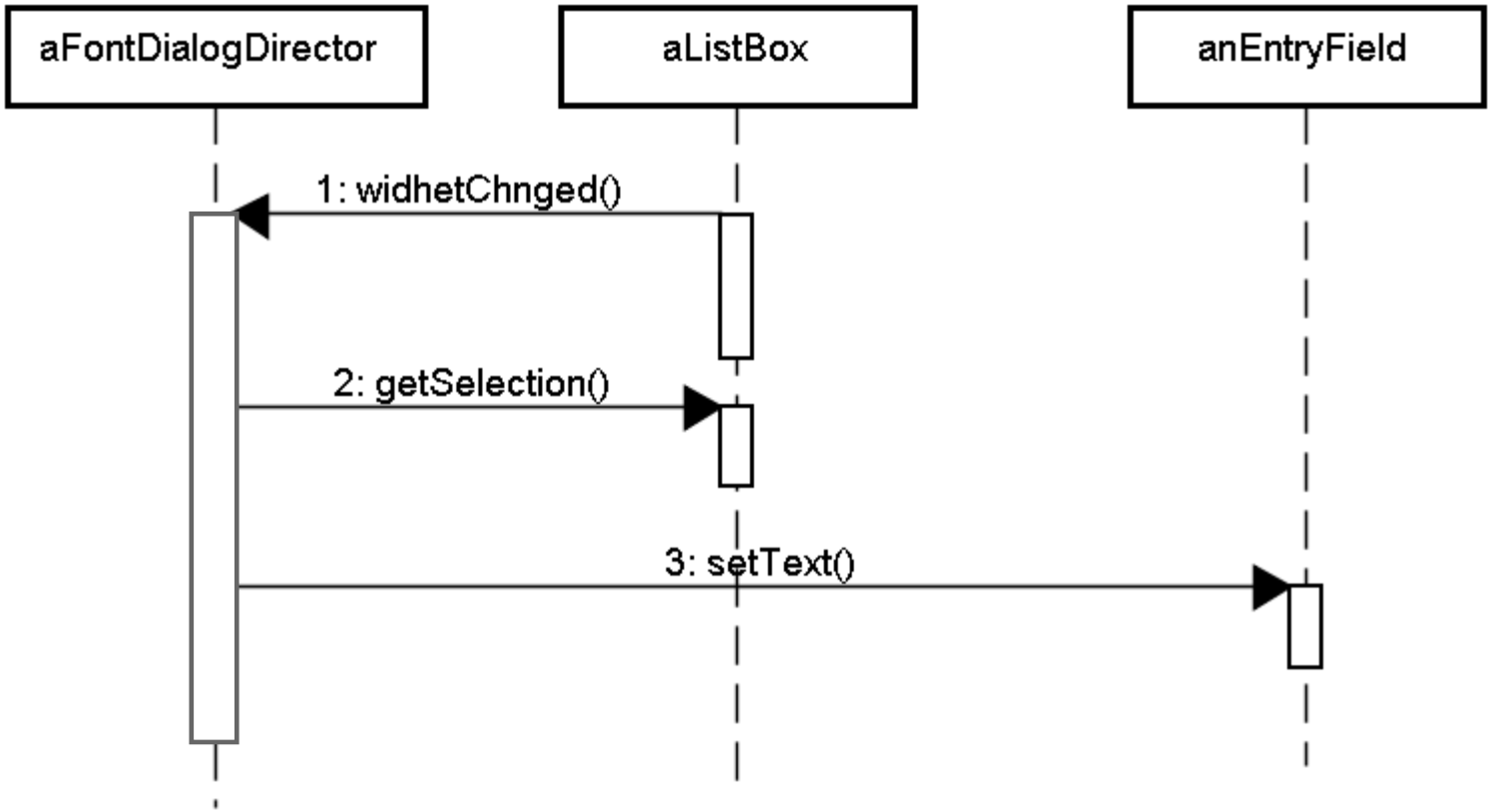




Mediator (behavioral)

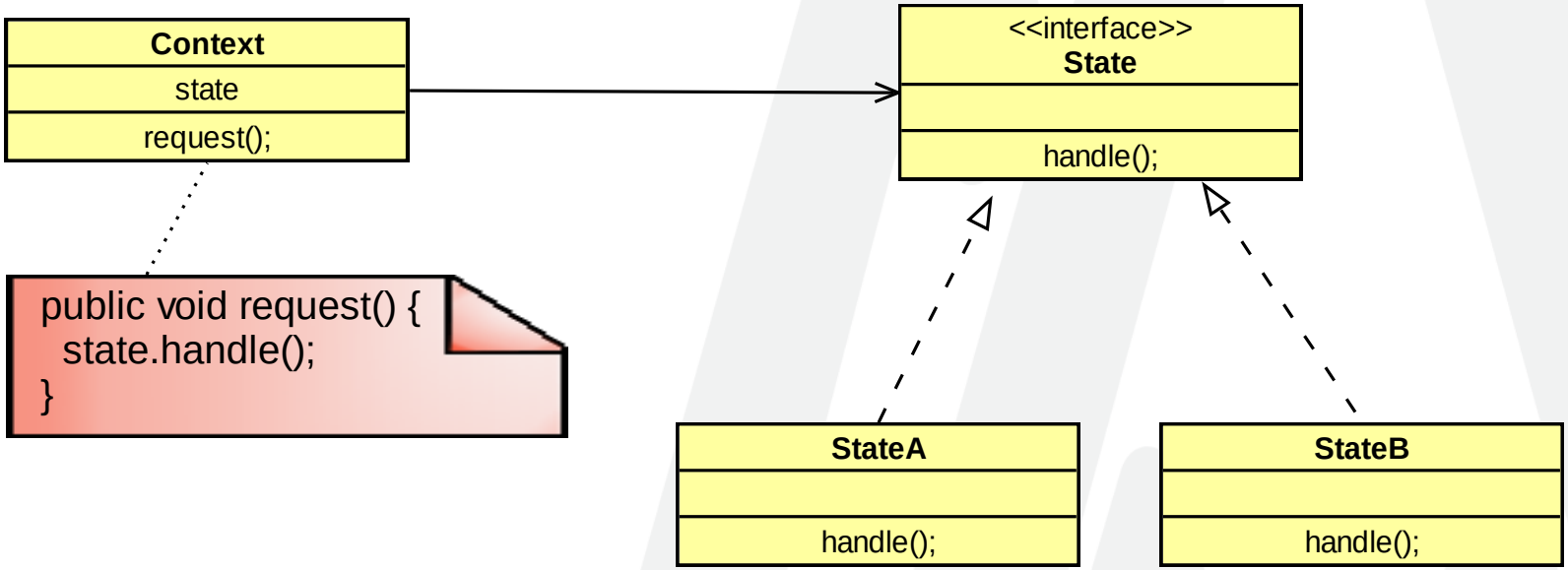


Mediator (behavioral)

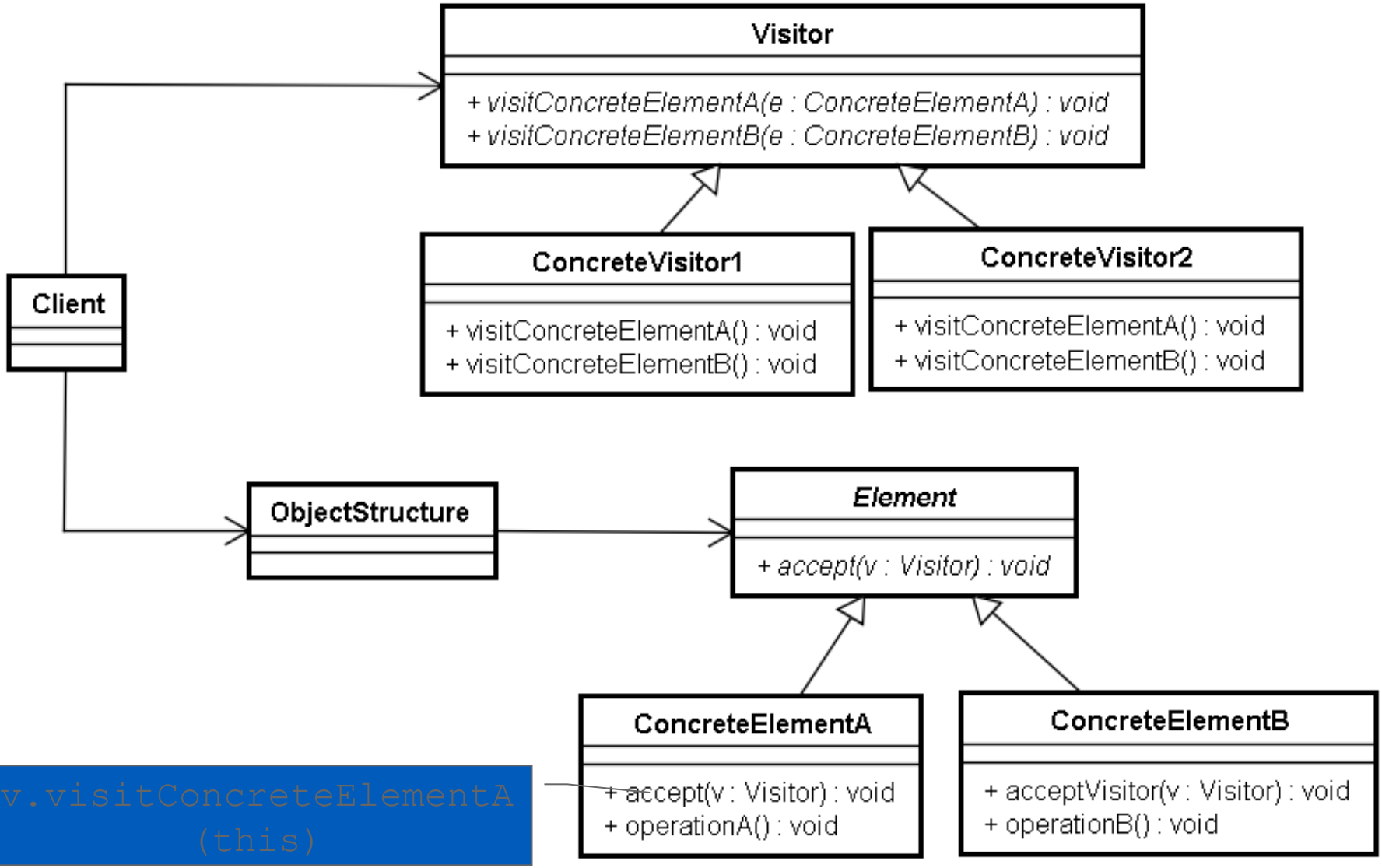


State (behavioral)

State Pattern allows an object to alter its behavior when its internal state changes.

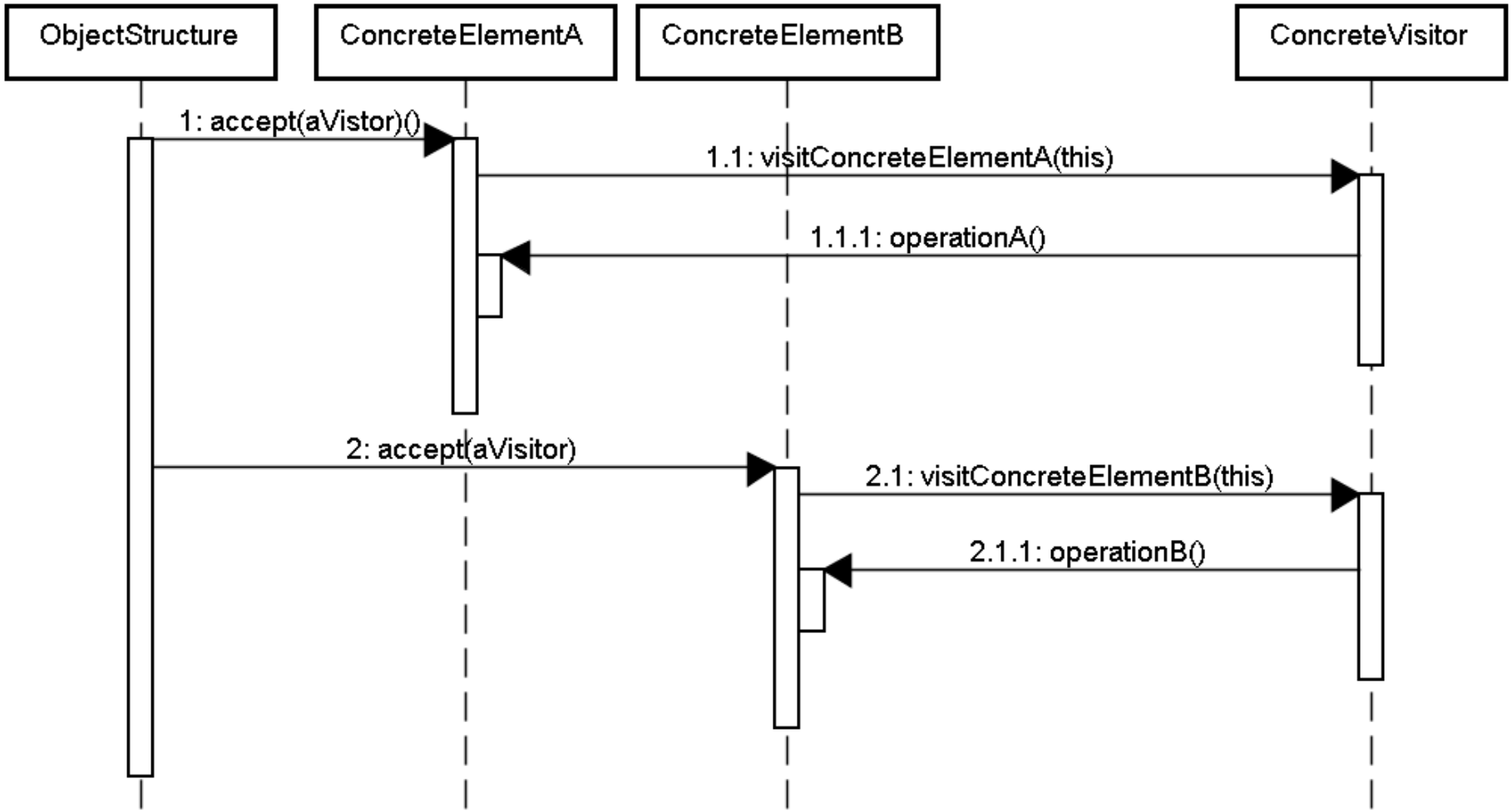


Visitor (behavioral)




```
v.visitConcreteElementA
(this)
```

Visitor (behavioral)



Visitor (behavioral)

- Visitor makes adding new operations **easy**
  simply by adding a new visitor

- A visitor gathers related operations and separates unrelated ones

- Adding new ConcreteElement classes is **hard**
  Is mostly likely to change the algorithm or the classes of objects that make up the structure?

- Visitor can accumulate state as they visit each element

Reference

https://cw.fel.cvut.cz/old/_media/courses/a4m33nms/designpatternsen.pptx